



GLOBAL PROGRAM IN MACHINE LEARNING AND ARTIFICIAL INTELLIGENCE

**SESSION : INTRODUCTION TO
NEURAL NETWORKS**

Instructor : SACHIN KUMAR
Master Trainer (UpGrad)

❖ Introduction

- Machine Learning & AI
- Problems faced by traditional ML algorithms
- Application on Neural Networks

❖ Perceptron – The building block

- What is a Perceptron?
- Activation functions

❖ Structure of an ANN & Feedforward in Neural Networks

- How does an ANN look like?
- Notations used for layers, input, outputs etc
- Different inputs & outputs to a Neural
- Feedforward in an Artificial Neural Network
- Types of Neural Networks

❖ Backpropagation in an ANN

- Training an ANN
- Cost Function for Regression/Classification
- Gradient Descent
- Feedforward and Backprop equations

❖ TensorFlow playground

- Building an ANN
- Training an ANN
- Changes in Parameters

❖ Doubt resolution

ARTIFICIAL INTELLIGENCE

Techniques that help computer mimic "cognitive" functions associated with the human mind,



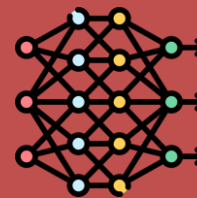
MACHINE LEARNING

Ability to learn pattern from data without being explicitly programmed



DEEP LEARNING

Extract pattern from data using Deep Neural Networks



Easy for computers to do:

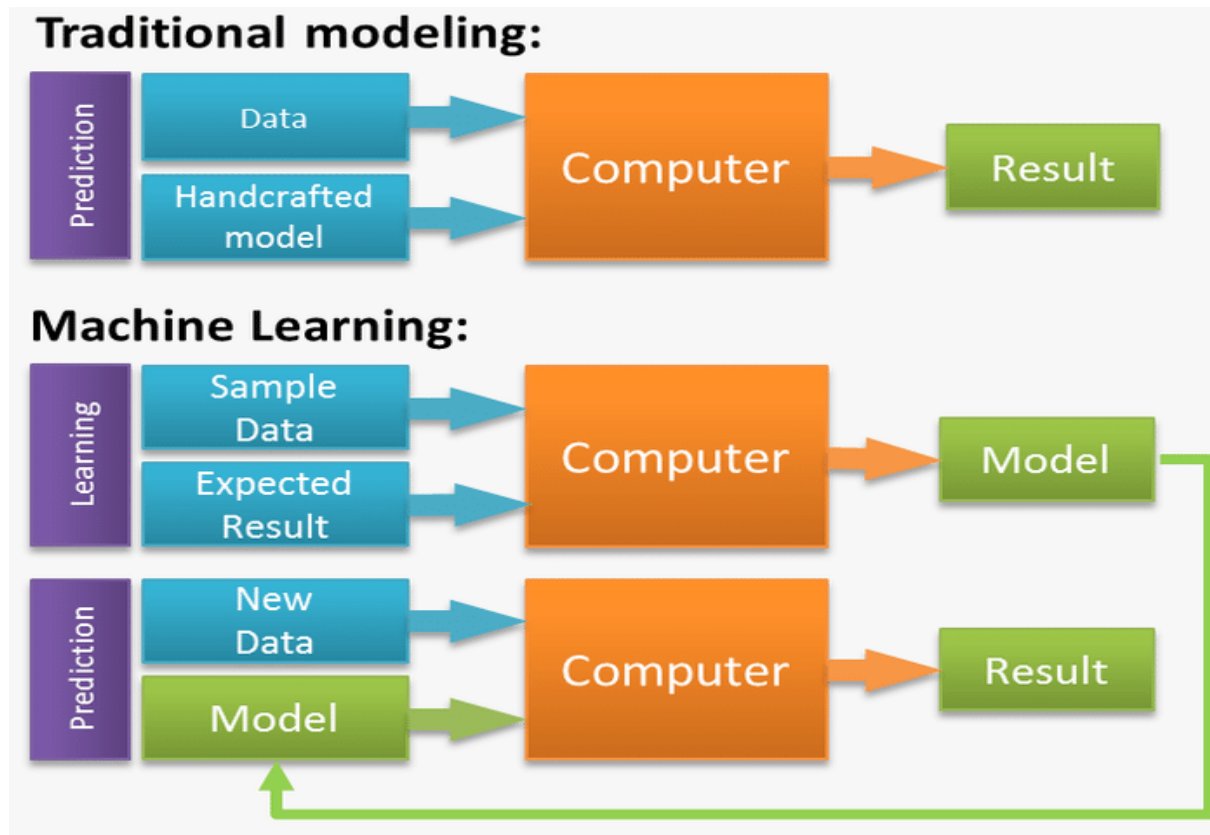
- ✓ Extensive mathematical computations
- ✓ Querying from a large corpus of text
- ✓ Combinatorial like Chess playing

Hard for computers to do:

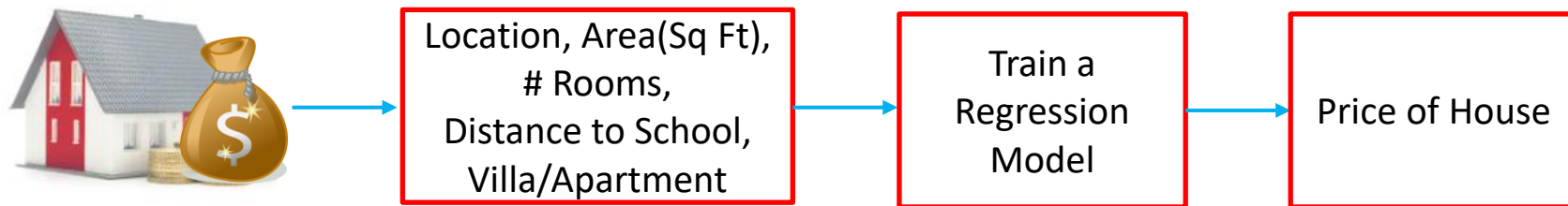
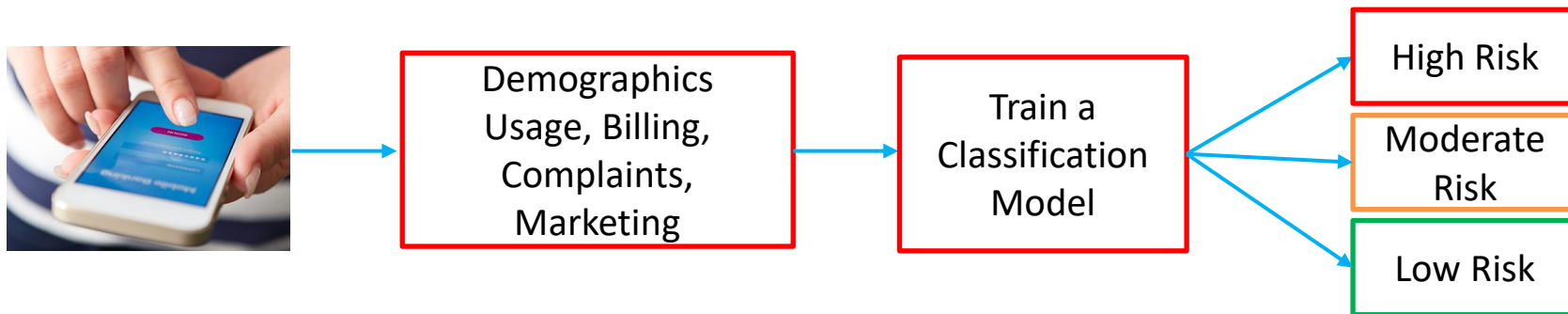
- Understand spoken language
- Recognize images and faces
- Recognize feelings
- Learning from few mistakes
- Generate New Ideas



Machine Learning was a Paradigm Shift



Machine Learning can Solve Many Problems





hand-crafted
Feature Extractor

"Simple" Trainable
Classifier

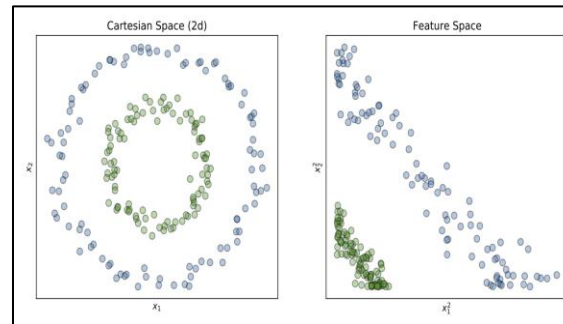
Car or
Truck?

The challenge to Machine Learning proved to be solving the tasks that are easy for people to ***perform***, but hard for people to ***describe*** formally.

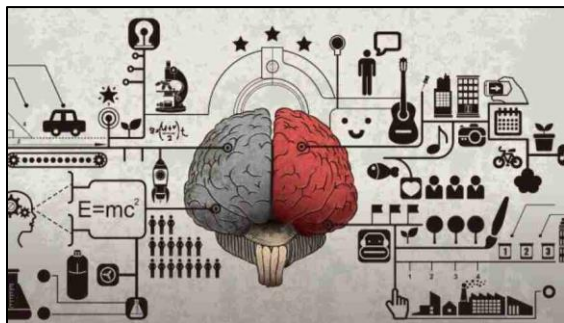
Challenges with Traditional Machine Learning



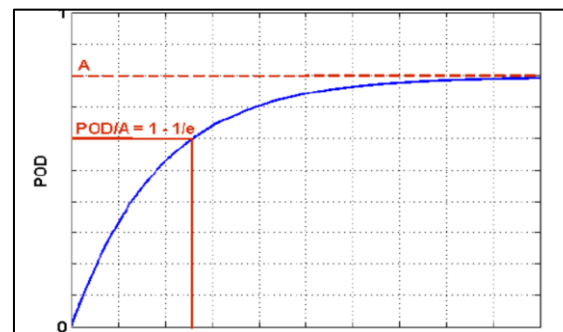
Require domain expertise to handcraft useful features



Results heavily depend on Feature Representation



Cannot solve extremely complex problems



Has an upper limit to Performance

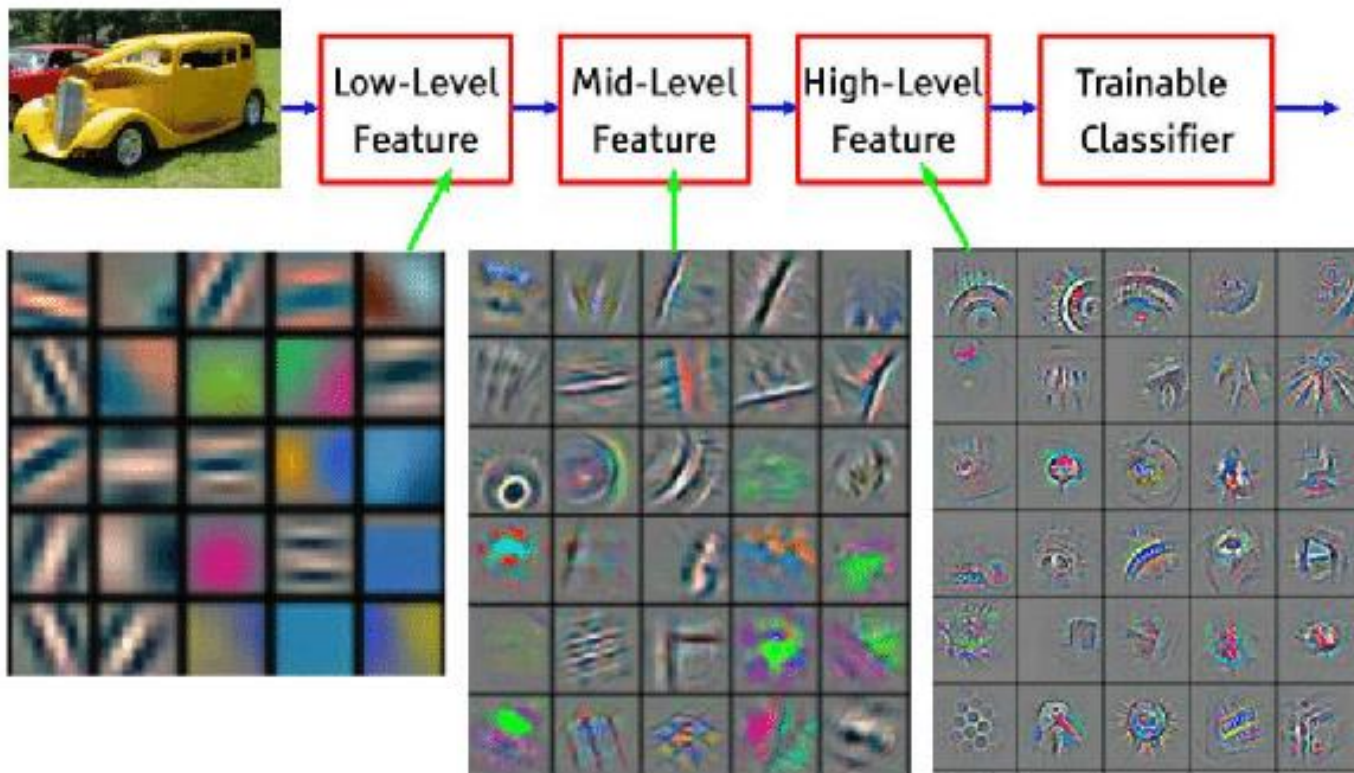
Learning hard-to-describe problem:

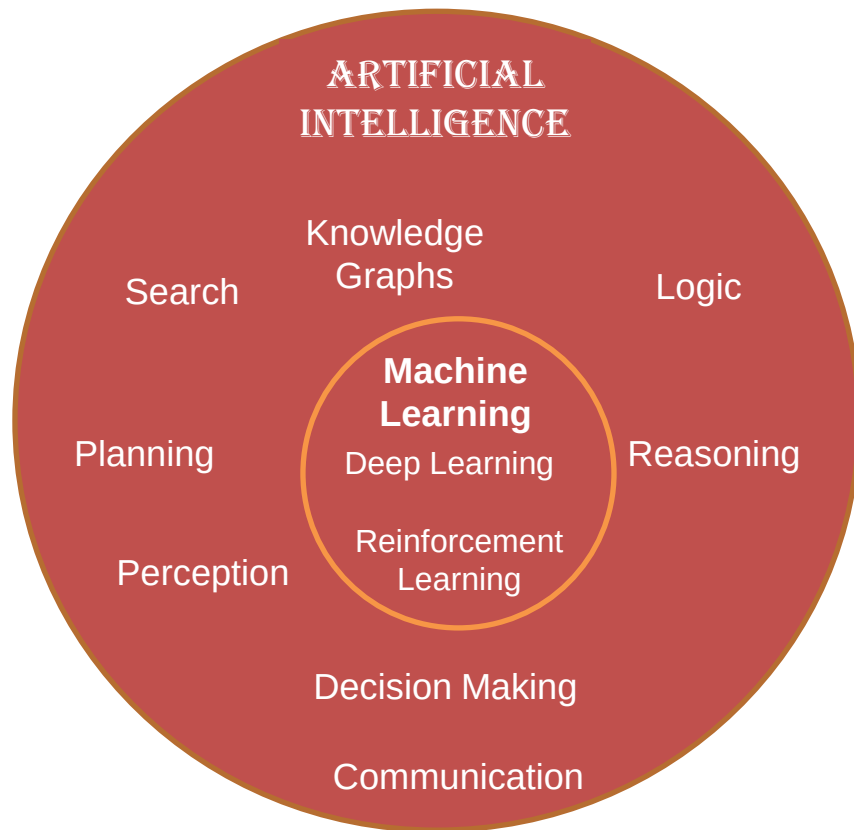
- Allow computers to learn from experience
- Understand the world in terms of a hierarchy of concepts
- Each concept defined through its relation to simpler concepts.

Advantages:

- No need for human operators to formally specify all the knowledge that the computer needs
- The hierarchy of concepts enables the computer to learn complicated concepts by building them out of simpler ones.

Deep Learning aka Hierarchical Feature Learning





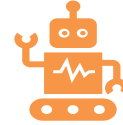
Ability of Machines to mimic "cognitive" functions that we associate with the human mind, such as "learning, seeing, interpreting and problem solving"



Computer
Vision



Speech
Recognition



Personal
Assistant



Medical
Diagnosis



Chatbots



Assisted driving
Cars



Text Data

Text Categorization

POS/NER/Semantics

Text Summarization

Text Generation

Language Translation



Image Data

Categorization

Character Recognition

Object Detection

Face Recognition

Image Generation



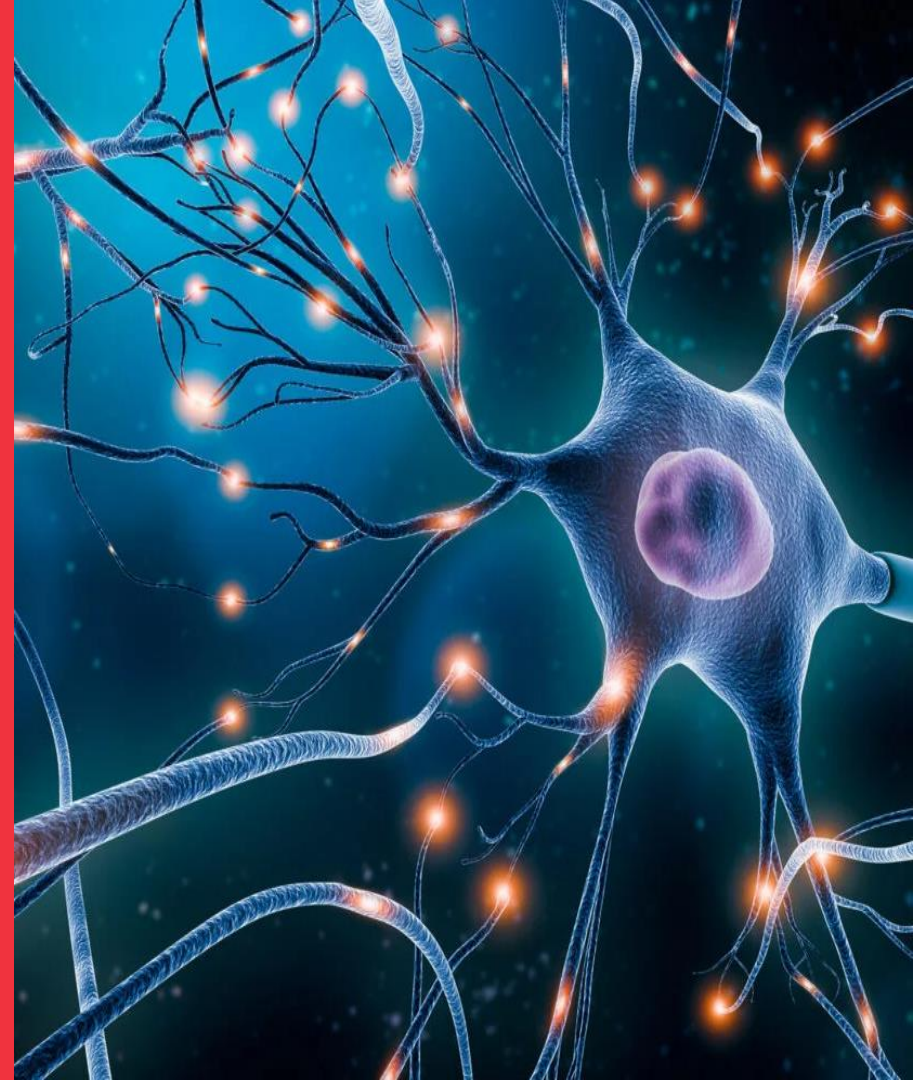
Voice Data

Speech to Text

Speech Synthesis

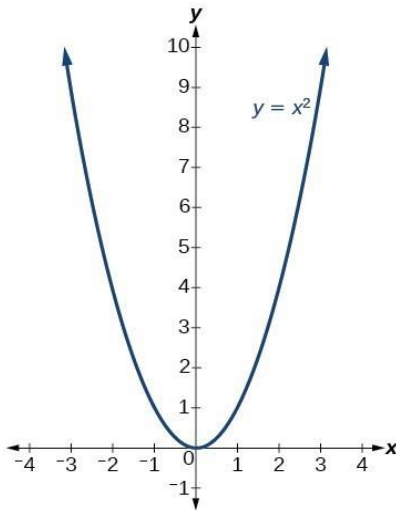
Music Recognition

Perceptron and Activation Functions



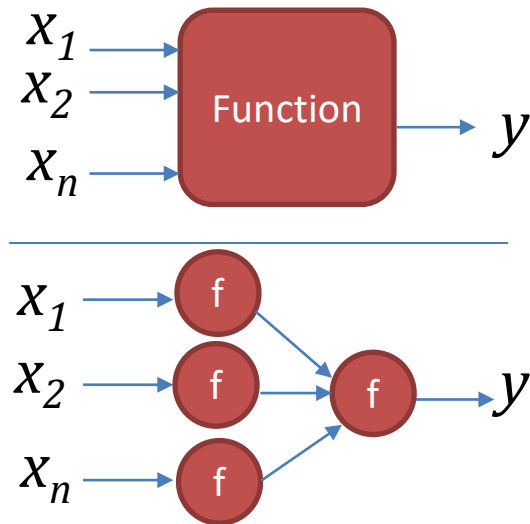
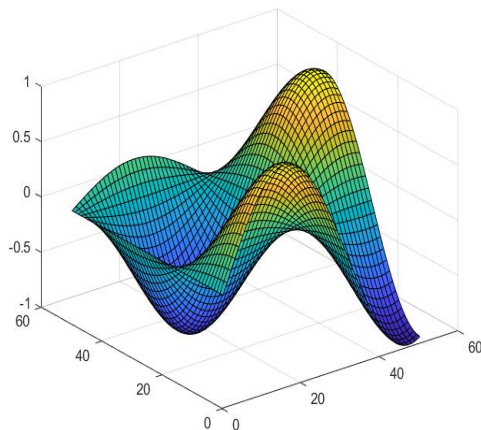
$$y = f(x)$$

$$f(x) = x^2$$

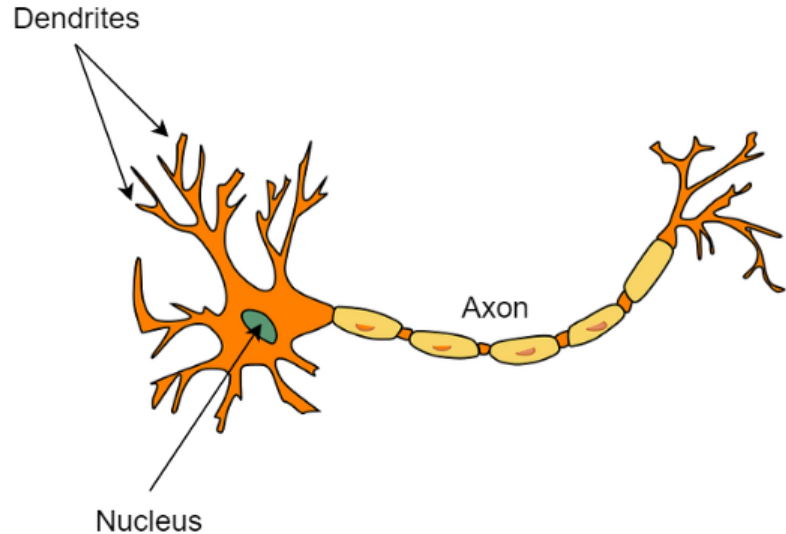


$$Y = f(W, X)$$

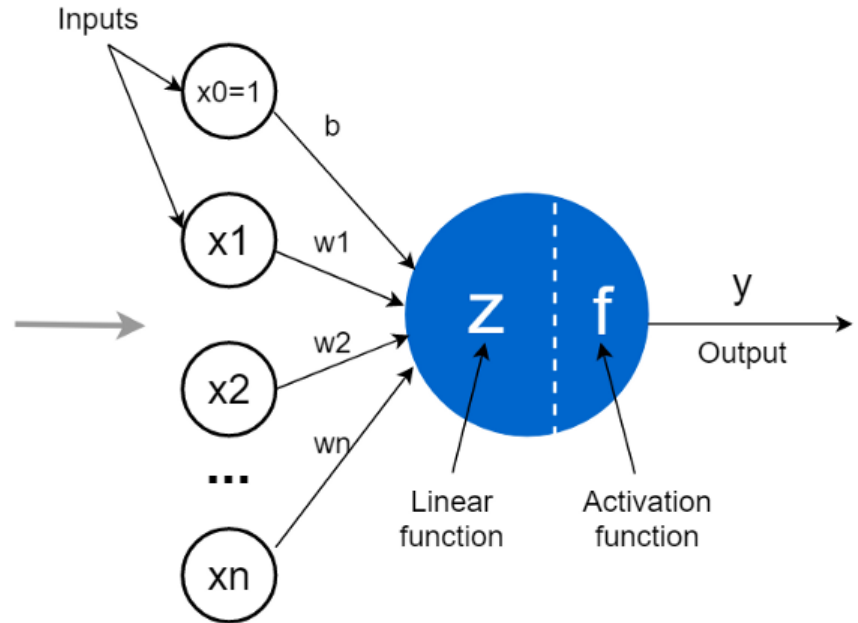
$$y_1 = w_1x_1 + w_2x_2 + \dots + w_nx_n$$



Artificial Neural Network – Inspired by Nature

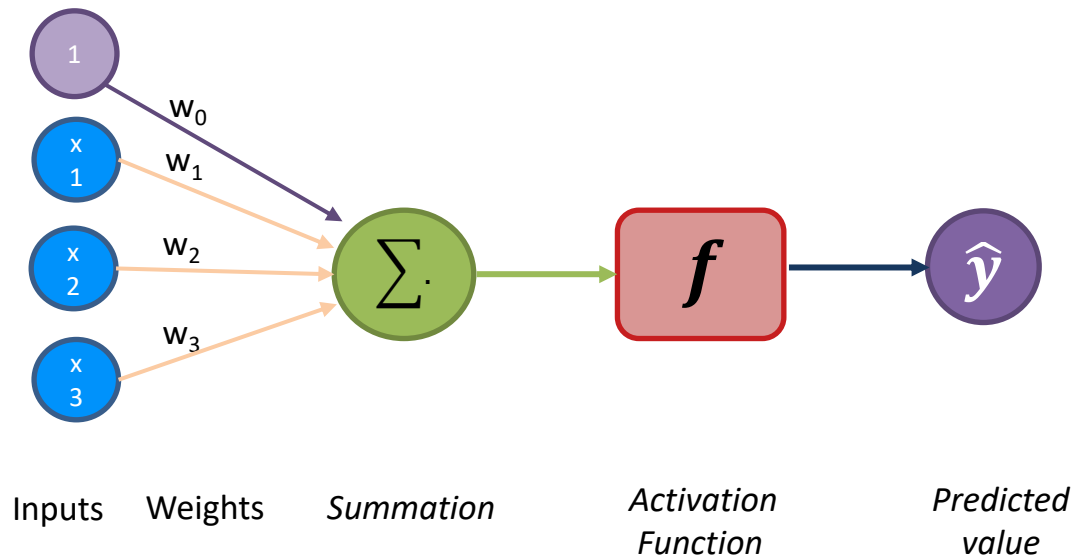


Biological Neuron



Artificial Neuron

A Closer Look At The Perceptron



*Non-linear
Function*

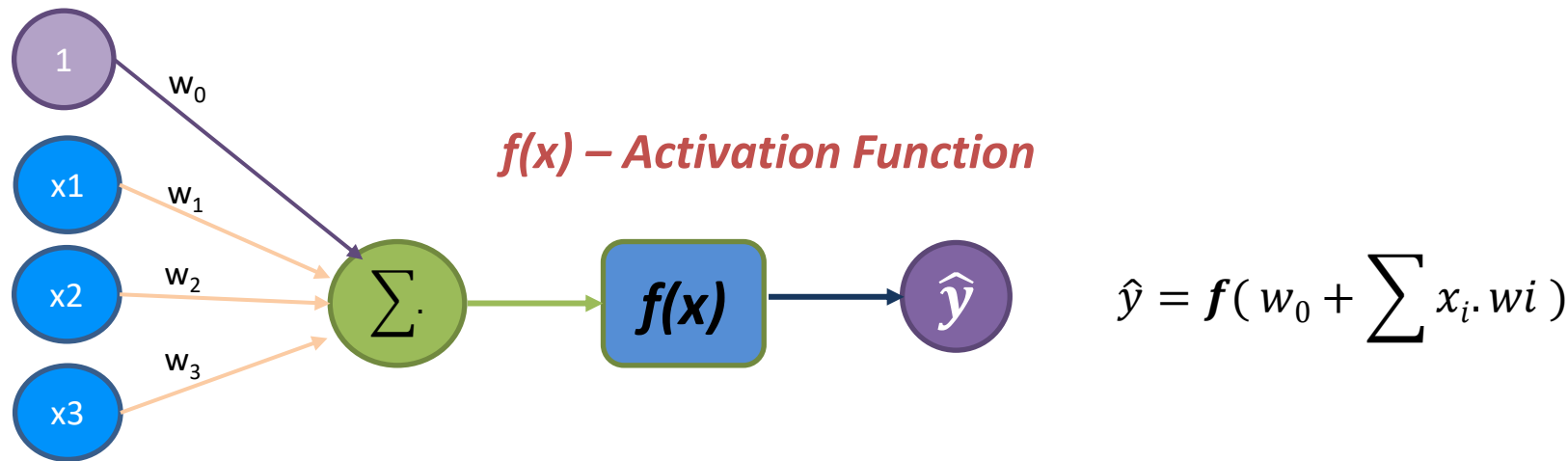
$$\hat{y} = f\left(\sum x_i \cdot w_i\right)$$

$$\hat{y} = f\left(w_0 + \sum x_i \cdot w_i\right)$$

$$\hat{y} = f(W^T \cdot X) \quad X = 1, x_1, x_2, \dots$$

FORWARD PROPOGATION

The Perceptron Can Take Many Forms



Let, $f(x) = 1$

$$Y = w_0 + w_1x_1 + w_2x_2 + w_3x_3$$

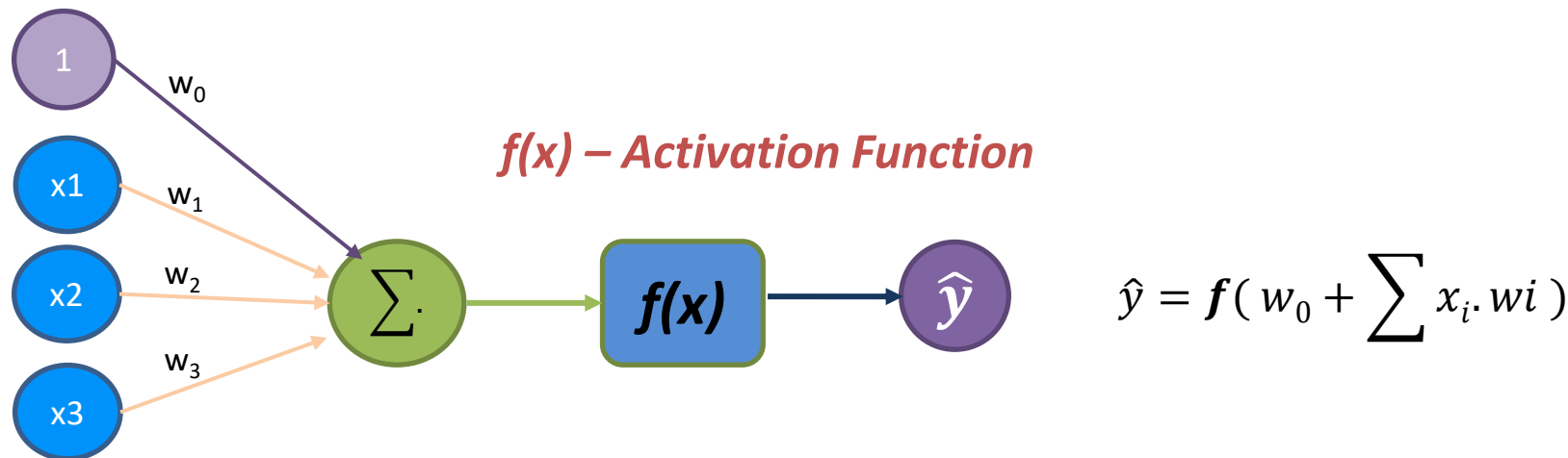
[Linear]

[Linear Regression]

$$w_0 = 1.2 \quad w_1 = 1.0 \quad w_2 = 1.25 \quad w_3 = 2.5$$
$$x_1 = 0.5, \quad x_2 = 0.3; \quad x_3 = 0.7$$

$$Y = 1.2 + 1 \cdot 0.5 + 1.25 \cdot 0.3 + 2.5 \cdot 0.7$$
$$= 3.825$$

The Perceptron Can Take Many Forms



Let, $f(x) = 1/(1 + e^{-x})$

$$Y = \frac{e^{(w_0 + w_1x_1 + w_2x_2 + w_3x_3)}}{1 + e^{(w_0 + w_1x_1 + w_2x_2 + w_3x_3)}}$$

[Sigmoid]

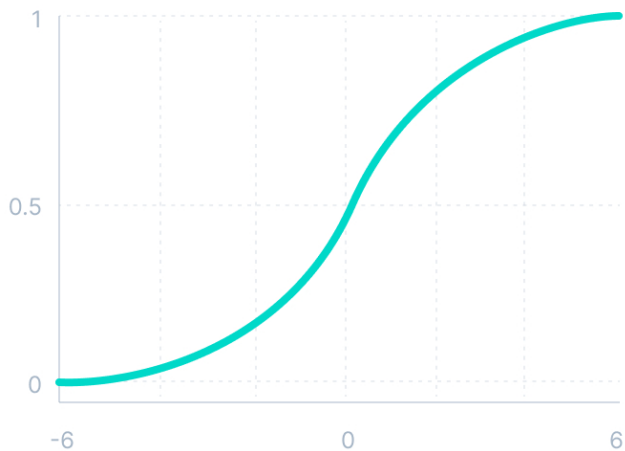
[Logistic Regression]

$$w_0 = 1.2 \quad w_1 = 1.0 \quad w_2 = 1.25 \quad w_3 = 2.5$$
$$x_1 = 0.5, \quad x_2 = 0.3; \quad x_3 = 0.7$$

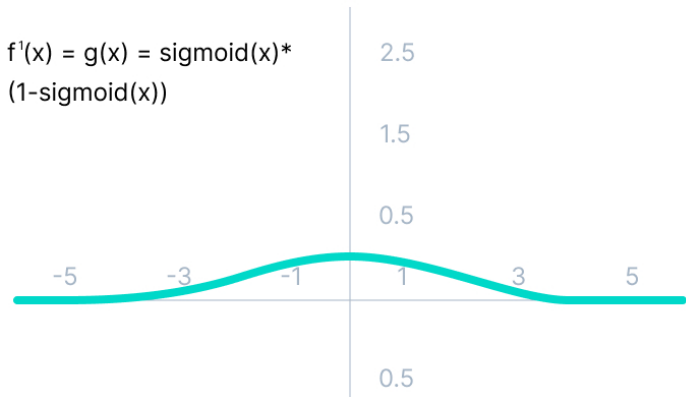
$$Y = \frac{e^{(3.825)}}{1 + e^{(3.825)}} = 0.978$$

$$f(x) = \frac{1}{1 + e^{-x}}$$

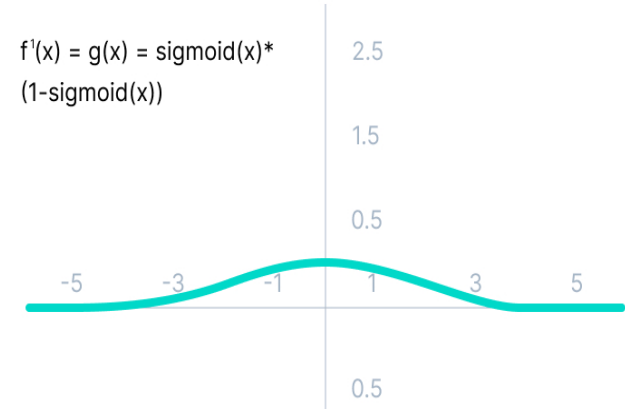
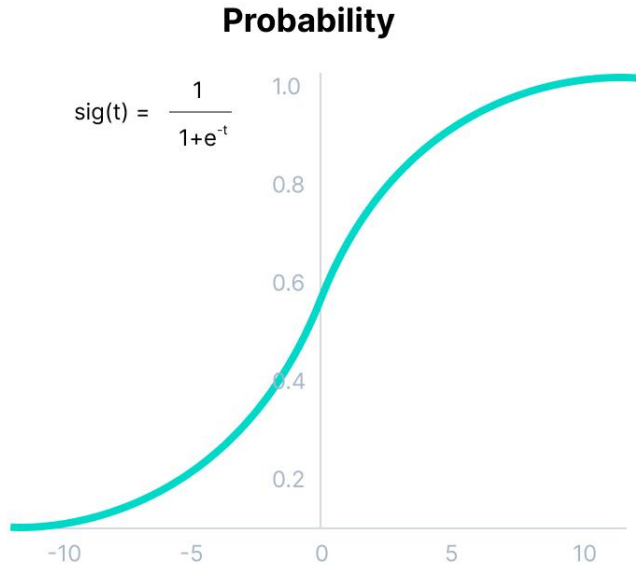
Sigmoid / Logistic



$$f'(x) = g(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$$

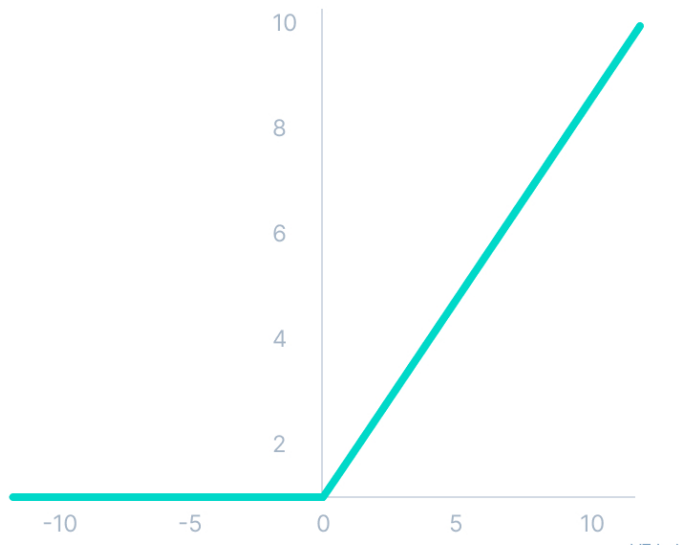


$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

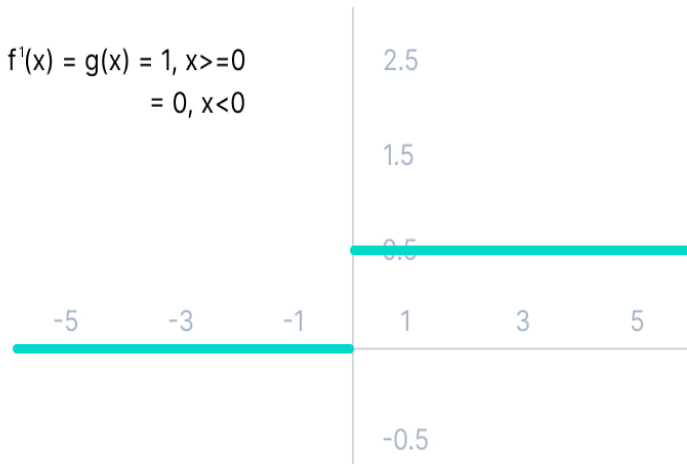


$$f(x) = \max(0, x)$$

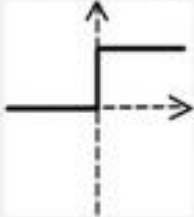
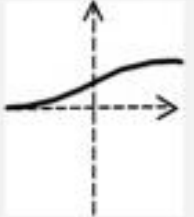
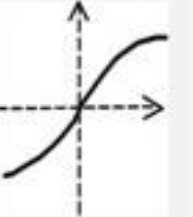
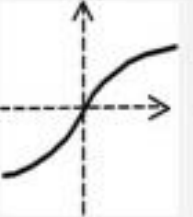
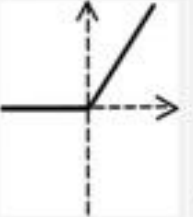
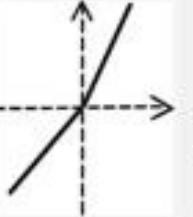
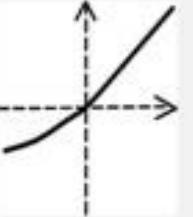
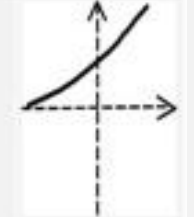
ReLU



$$f'(x) = g(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

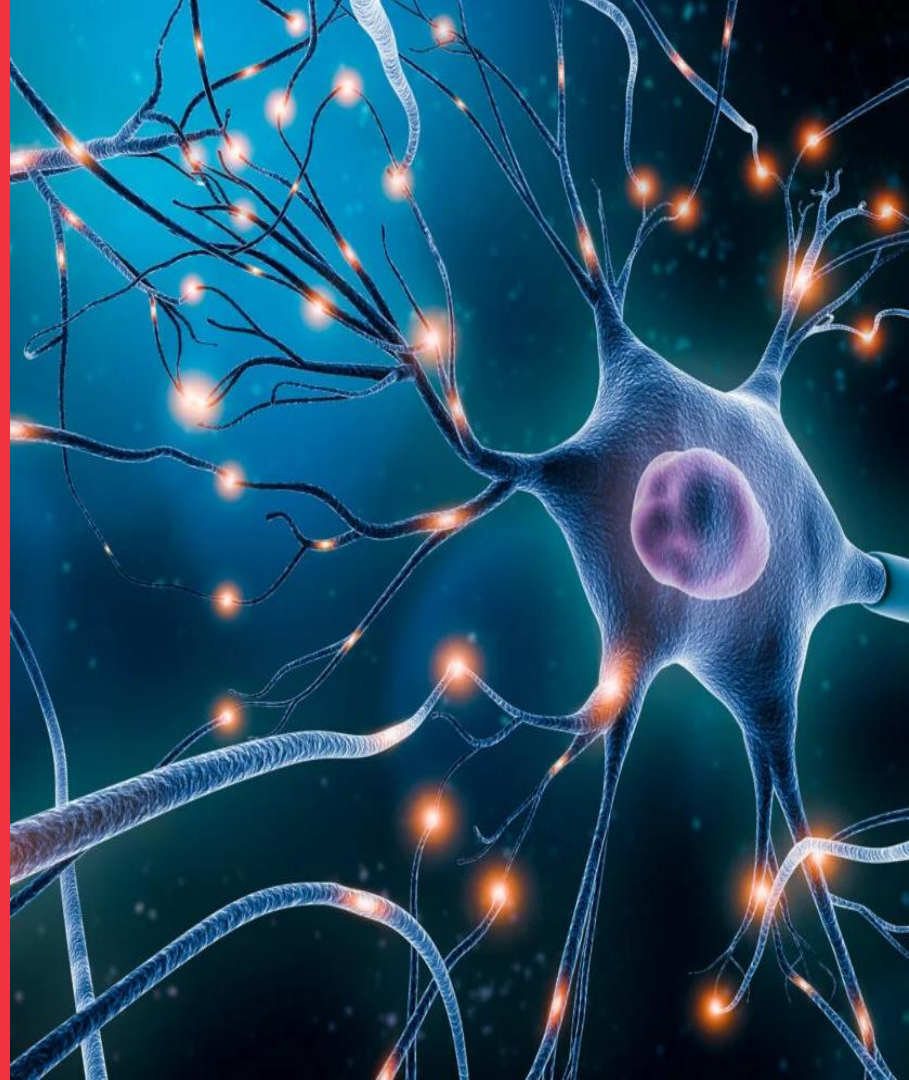


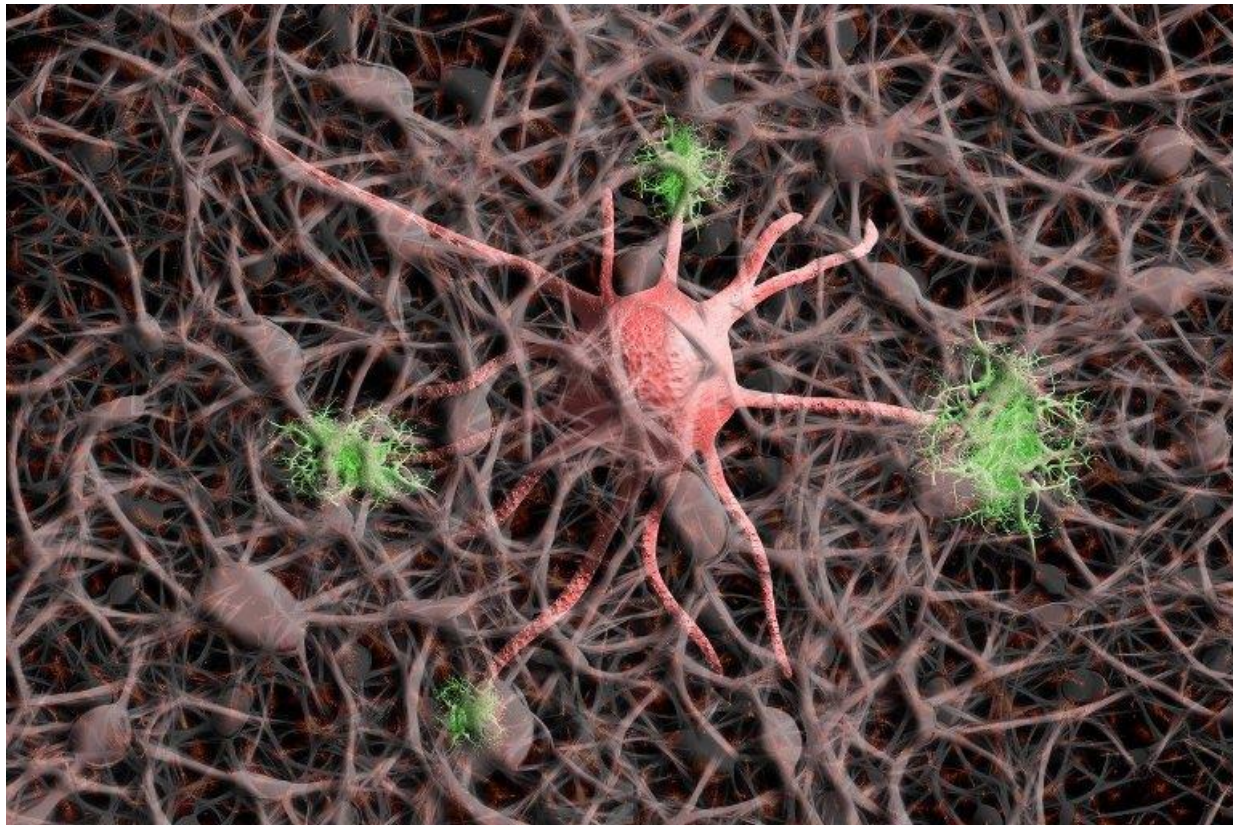
Other Activation Functions

Binary Step	Logistic	TanH	ArcTan	ReLU	PreLU	ELU	SoftPlus
							
$f(x)$	$f(x)$	$f(x)$	$f(x)$	$f(x)$	$f(x)$	$f(x)$	$f(x)$
$=$	$=$	$= \tanh(x)$	$=$	$=$	$=$	$=$	$=$
$\begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$\frac{1}{1 + e^{-x}}$	$\frac{2}{1 + e^{-2x}} - 1$	$\tan^{-1}(x)$	$\begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$\begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$\begin{cases} \alpha(e^{-x}-1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$\log_e(1 + e^x)$
$f'(x)$	$f'(x)$	$f'(x)$	$f'(x)$	$f'(x)$	$f'(x)$	$f'(x)$	$f'(x)$
$=$	$=$	$=$	$=$	$=$	$=$	$=$	$=$
$\begin{cases} 0 & \text{for } x \neq 0 \\ \delta & \text{for } x = 0 \end{cases}$	$f(x)(1-f(x))$	$1-f(x)^2$	$\frac{1}{x^2 + 1}$	$\begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$\begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$\begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$\frac{1}{1 + e^{-x}}$

Artificial Neural Networks

Intro to NN - Sachin Kumar

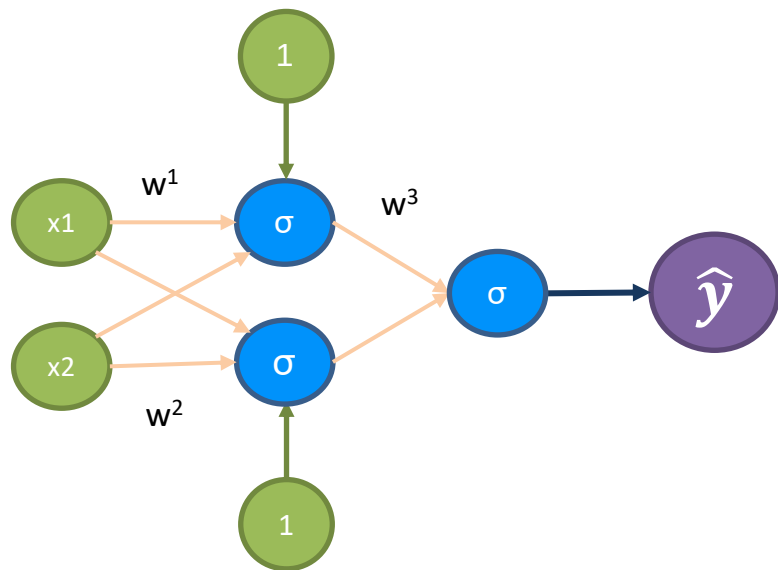




Human Brain:

- 60% Fat
- 100 Billion Neurons
- 1000 trillion Synapses – connections
- Connection store memory and process information.

A Simple Multi-Layered Perceptron



$$x_1, x_2$$

} Input

$$\alpha_1 = \sigma(w_i x_i + b_1)$$

$$\alpha_2 = \sigma(w_i x_i + b_2)$$

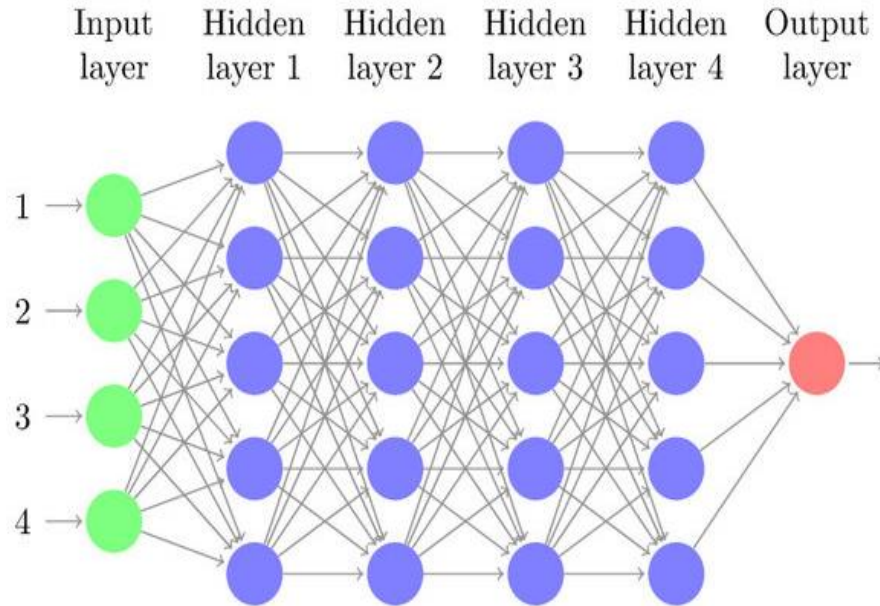
} Intermediate

$$\hat{y} = \sigma(w^3_1 \alpha_1 + w^3_2 \alpha_2)$$

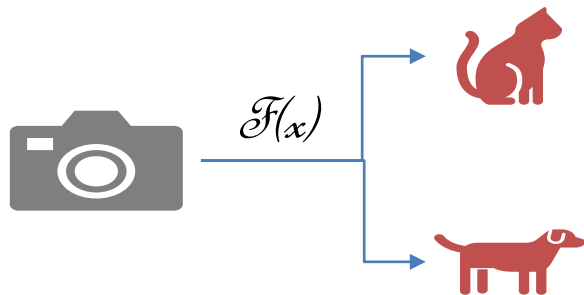
} Output

Multi-layered perceptron sum together the simple linear function, and each output a non linear function. The more layers we have in our hidden layer, the more complex non linear models we can find.

MLP – Universal Function Approximator

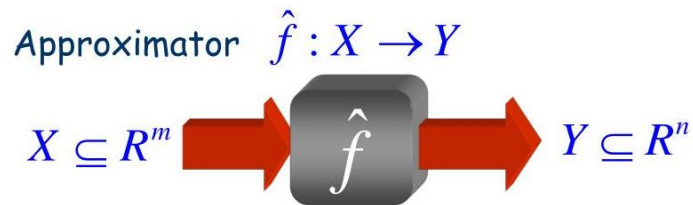
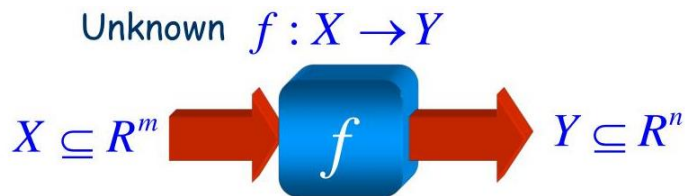


- Multi Layer Perceptron – 3 or more layers of nonlinearly-activating nodes.
- MLPs are fully connected.
 - Node $i \rightarrow w * \text{Node } j$



Task: Find the function $F(x)$ that can classify the image into Dog or Cat.

Use of Function Approximator: A technique for estimating an unknown underlying function using historical or available observations from the domain. Function approximations are used where theoretical models are unavailable or hard to compute.



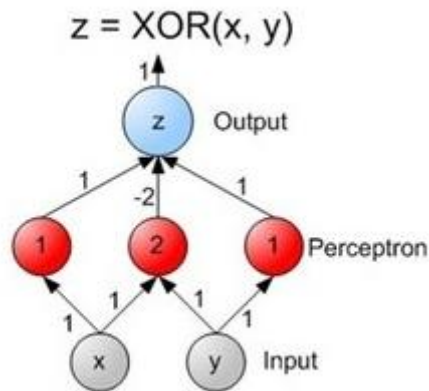
Taylor Series:
$$f(x) = f(a) + f'(a) \frac{(x-a)}{1!} + f''(a) \frac{(x-a)^2}{2!} + f'''(a) \frac{(x-a)^3}{3!} + \dots$$
$$= \sum_{n=0}^{\infty} \frac{f^n(a)}{n!} (x-a)^n$$

Fourier Series:
$$f(t) = \frac{a_0}{2} + \sum_{n=1}^{\infty} [a_n \cos(n\omega t) + b_n \sin(n\omega t)] \text{ or,}$$

(Periodic Functions)

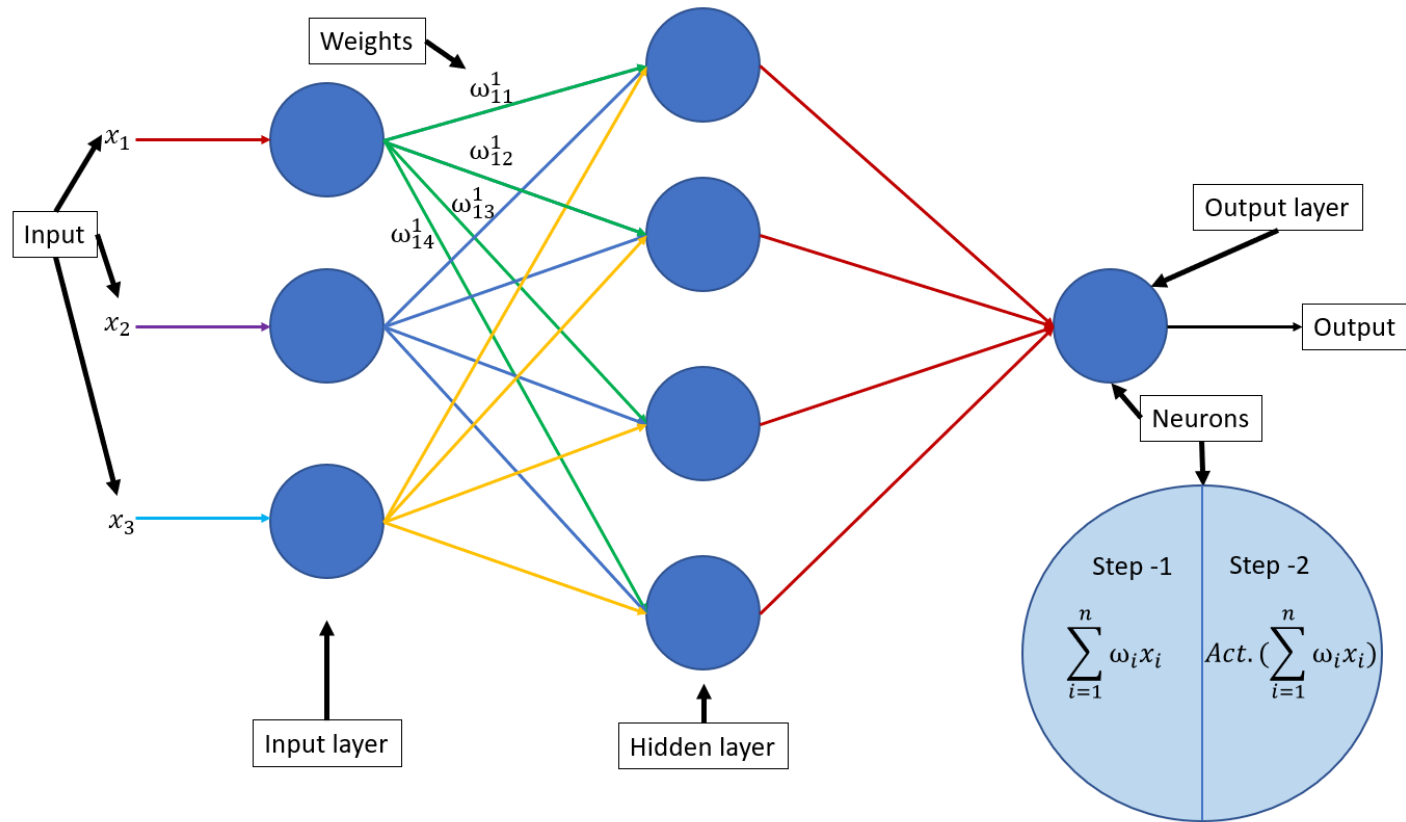
$$f(t) = \frac{1}{2} a_0 + a_1 \cos(\omega t) + a_2 \cos(2\omega t) + \dots$$
$$+ b_1 \sin(\omega t) + b_2 \sin 2(\omega t) + \dots$$

Multi-Layer Perceptron:



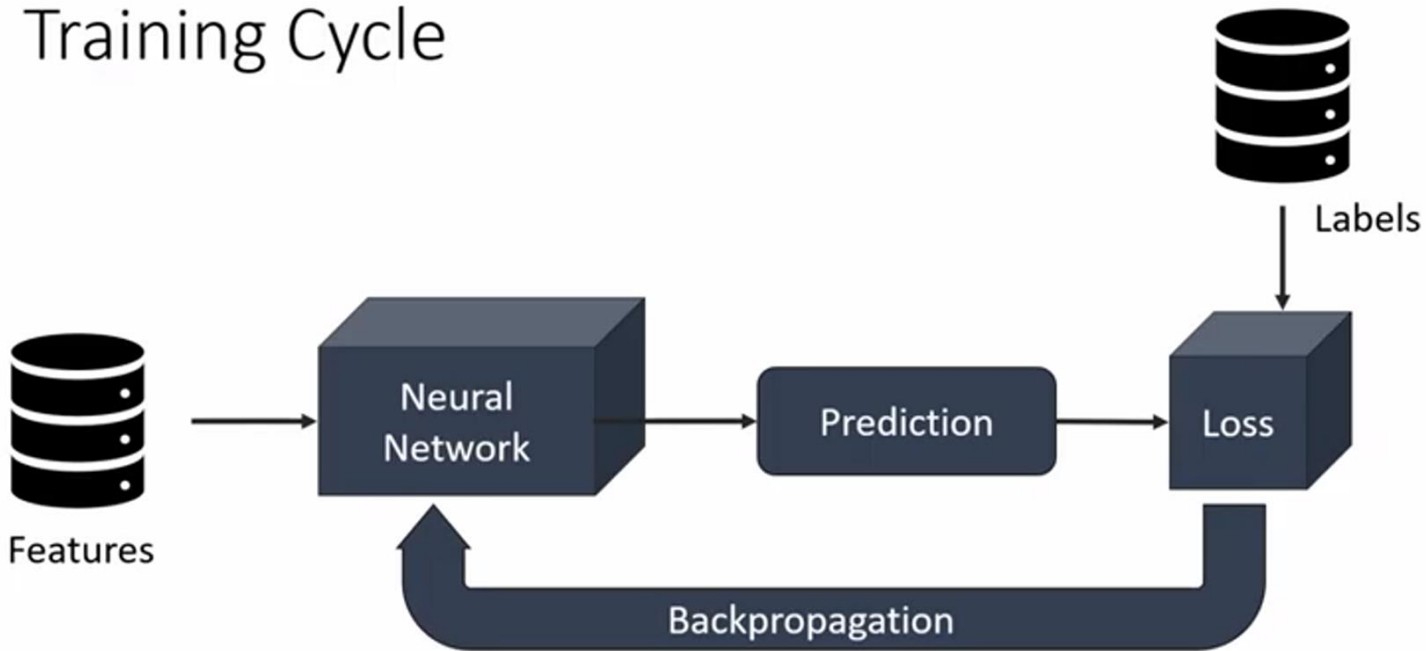
- MLPs are also function approximators.
- **Cybenko's theorem** : Given any known/unknown function, there exists a MLP that can approximate the function to an arbitrary degree of precision.
 - The network can be 1 layered with unbounded no of neurons
 - The network can be multi-layered with bounded no of neurons.
- Learning = $\delta(\text{connection weights})$ after each piece of data is processed.
- MLP learn **hierarchical feature** representations. Do not require feature engineering.
- **MLPs are Universal Function Approximator that can learn from the data !!**

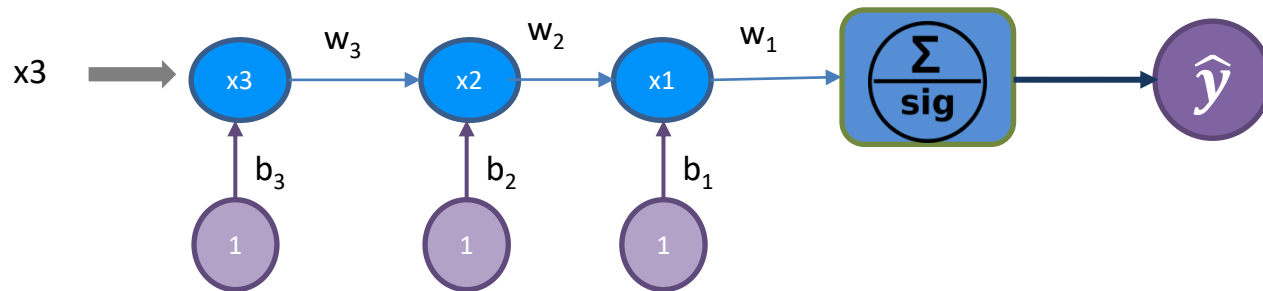
Topology of Artificial Neural Network



- The neurons in an ANN are arranged in layers, and these layers are arranged sequentially (from Input to Output).
- Every neuron in the neural network has a bias value associated with it, and each interconnection has a weight associated with it.
- Neurons are densely connected, i.e., all neurons in layer L are connected to all neurons in layer $L+1$.
- The neurons within the same layer **do not** interact with each other.
- All neurons in a particular hidden layer use the same activation function. Different hidden layers can use different activation functions.

Training Cycle





1

Define the Network

$$\hat{y} = f(w_1x_1 + b_1)$$

$$x_1 = (w_2x_2 + b_2)$$

$$x_2 = (w_3x_3 + b_3)$$

2

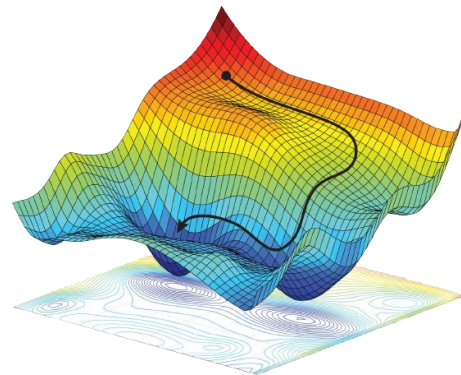
Define the Cost Function

$$L(w): \sum (y - \hat{y})^2$$

- Quantifies model performance
- # Continuous and mostly differentiable

3

Learn the weights/biases



- Gradient descent is an optimization algorithm used to find the values of parameters (coefficients) of a function (f) that minimizes a cost function (cost).
- Gradient descent is best used when the parameters cannot be calculated analytically .
- It finds the parameters using an iterative method:
 - The goal is to continue to try different values for the coefficients, evaluate their cost and select new coefficients that have a slightly better (lower) cost.
 - Repeating this process enough times will lead to the coefficients that result in the minimum cost.

Gradient descent :

1. Estimate the error using current weights (randomly initialized)
2. Find the derivative (gradient) of the cost function.
3. Move in the direction opposite of the gradient, until convergence.

Gradient Descent Update Rule:

repeat until convergence {

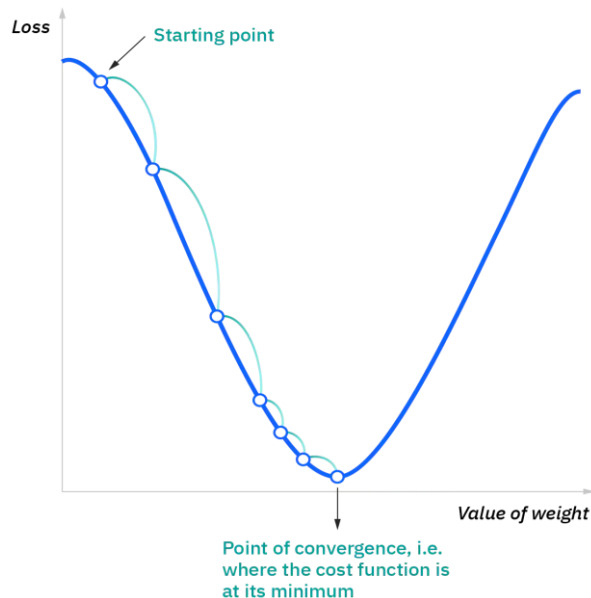
$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

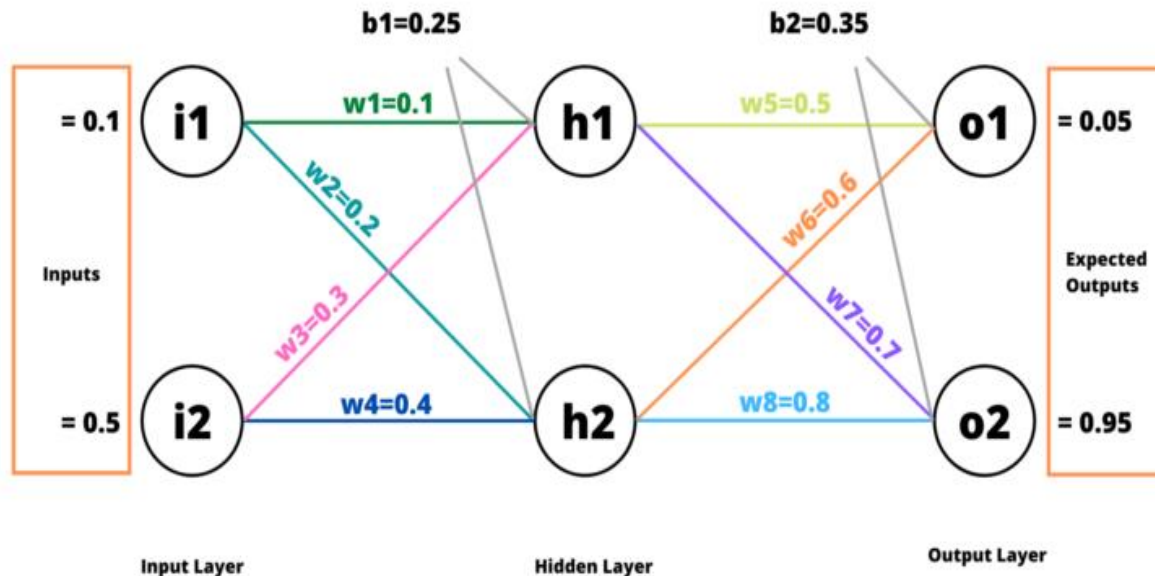
(for $j=1$ to $j=k$)

}

α – learning rate

j - features



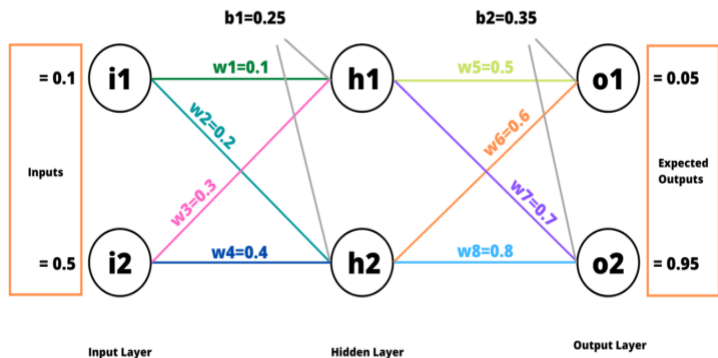


Find the weights ($w_1, w_2, w_3 \dots$) such that the cost (error) is minimized

$$E(w) \text{ or } L(w): \sum (y - \hat{y})^2$$

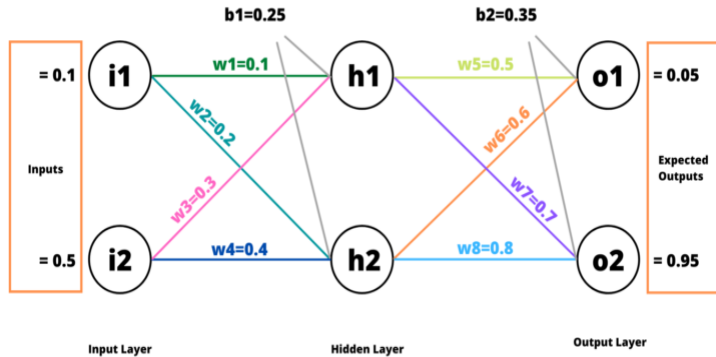
Find the weights (w_1, w_2, w_3) such that the cost (error) is minimized

$$L(w): \sum (y - \hat{y})^2$$



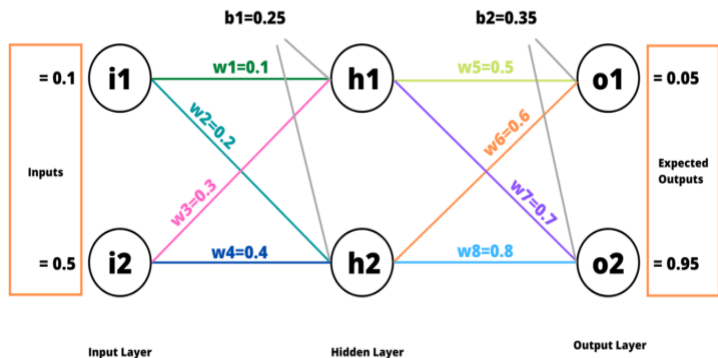
1. Start with random weights ($w_1, w_2, w_3, \dots$)
2. Calculate the values at each node and the output.
3. Calculate the Cost Function (Error)
4. Find the Derivative of Cost Function w.r.t each weight.
5. Change w_j such that
$$w_j =: w_j - \alpha * \frac{\partial L}{\partial w_j}$$
6. Stop when values converge.

1. Start with random weights ($w_1, w_2, w_3, \dots$)



$w1 = 0.1$
 $w2 = 0.2$
 $w3 = 0.3$
 $w4 = 0.4$
 $w5 = 0.5$
 $w6 = 0.6$
 $w7 = 0.7$
 $w8 = 0.8$

2. Calculate the values at each node and the output. [Forward Pass]



$$sum_{h1} = i_1 * w_1 + i_2 * w_3 + b_1 \quad output_{h1} = \frac{1}{1 + e^{-sum_{h1}}}$$
$$sum_{h1} = 0.1 * 0.1 + 0.5 * 0.3 + 0.25 = 0.41 \quad output_{h1} = \frac{1}{1 + e^{-0.41}} = 0.60108$$

$$sum_{h2} = i_1 * w_2 + i_2 * w_4 + b_1 = 0.47$$

$$output_{h2} = \frac{1}{1 + e^{-sum_{h2}}} = 0.61538$$

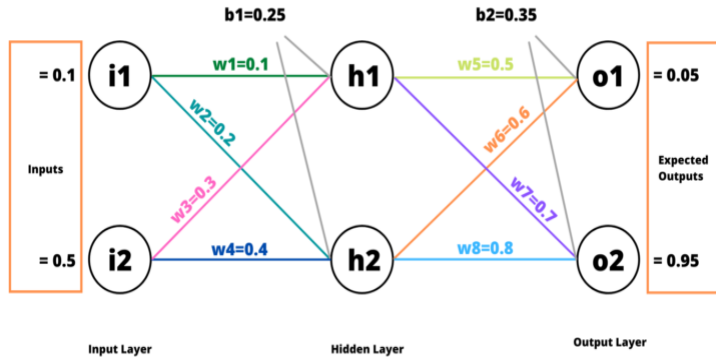
$$sum_{o1} = output_{h1} * w_5 + output_{h2} * w_6 + b_2 = 1.01977$$

$$output_{o1} = \frac{1}{1 + e^{-sum_{o1}}} = 0.73492$$

$$sum_{o2} = output_{h1} * w_7 + output_{h2} * w_8 + b_2 = 1.26306$$

$$output_{o2} = \frac{1}{1 + e^{-sum_{o2}}} = 0.77955$$

3. Calculate the Cost Function (Error)



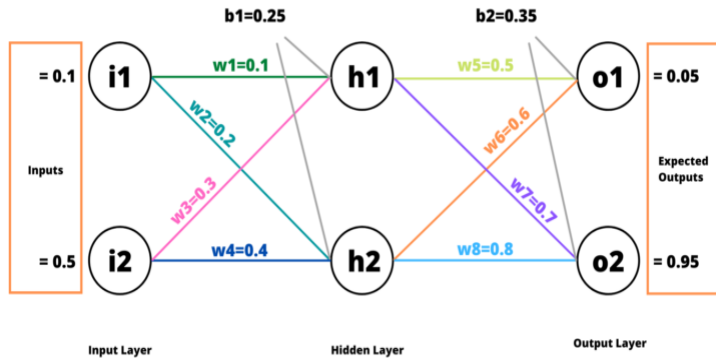
$$E_{total} = \sum \frac{1}{2} (target - output)^2$$

$$E_1 = \frac{1}{2} (0.05 - 0.73492)^2 = 0.23456$$

$$E_2 = \frac{1}{2} (0.95 - 0.77955)^2 = 0.01452$$

$$E_{total} = E_1 + E_2 = 0.24908$$

4. Find the Derivative of Cost Function w.r.t each weight.

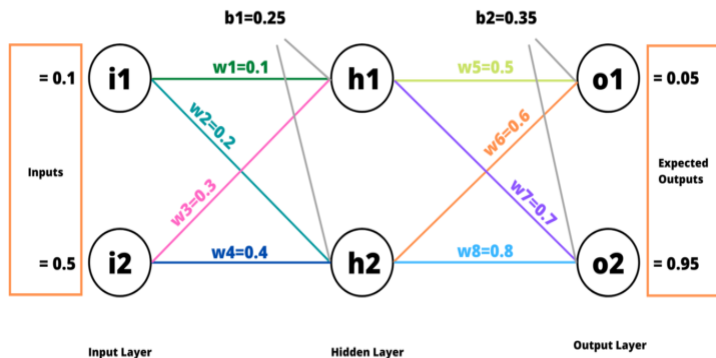


$E1$ is affected by $output_{o1}$, $output_{o1}$ is affected by sum_{o1} , and sum_{o1} is affected by $w5$.

So by Calculus Chain Rule.

$$\frac{\partial E_{total}}{\partial w5} = \frac{\partial E_{total}}{\partial output_{o1}} * \frac{\partial output_{o1}}{\partial sum_{o1}} * \frac{\partial sum_{o1}}{\partial w5}$$

BACK PROPAGATION



4. Find the Derivative of Cost Function w.r.t each weight.

$$\frac{\partial E_{total}}{\partial w5} = \frac{\partial E_{total}}{\partial output_{o1}} * \frac{\partial output_{o1}}{\partial sum_{o1}} * \frac{\partial sum_{o1}}{\partial w5}$$

$$E_{total} = \frac{1}{2}(target_1 - output_{o1})^2 + \frac{1}{2}(target_2 - output_{o2})^2$$

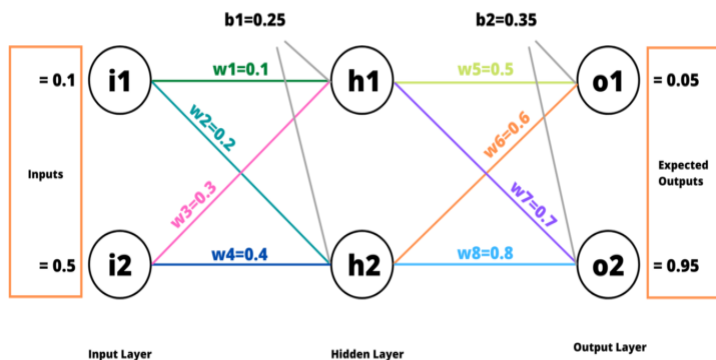
$$\frac{\partial E_{total}}{\partial output_{o1}} = 2 * \frac{1}{2} * (target_1 - output_{o1}) * -1 = output_{o1} - target_1$$

$$\frac{\partial output_{o1}}{\partial sum_{o1}} = output_{o1}(1 - output_{o1}) \quad \frac{\partial sum_{o1}}{\partial w5} = output_{h1}$$

$$\frac{\partial E_{total}}{\partial w5} = [output_{o1} - target_1] * [output_{o1}(1 - output_{o1})] * [output_{h1}]$$

$$\frac{\partial E_{total}}{\partial w5} = 0.68492 * 0.19480 * 0.60108 = 0.082$$

5. Change the weights in negative direction of gradient.

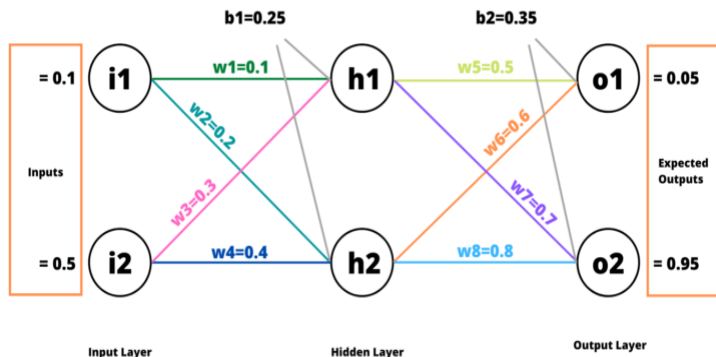


$$new_w_5 = w_5 - n * \frac{\partial E_{total}}{\partial w_5}, \text{ where } n \text{ is learning rate.}$$

$$new_w_5 = 0.5 - 0.6 * 0.08020$$

$$new_w_5 = 0.45187$$

We can proceed similarly for all other weights w6, w7, and w8.



5. Change the weights in negative direction of gradient.

For weights in the hidden layer (w_1, w_2, w_3, w_4)

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_1}{\partial w_1} + \frac{\partial E_2}{\partial w_1}.$$

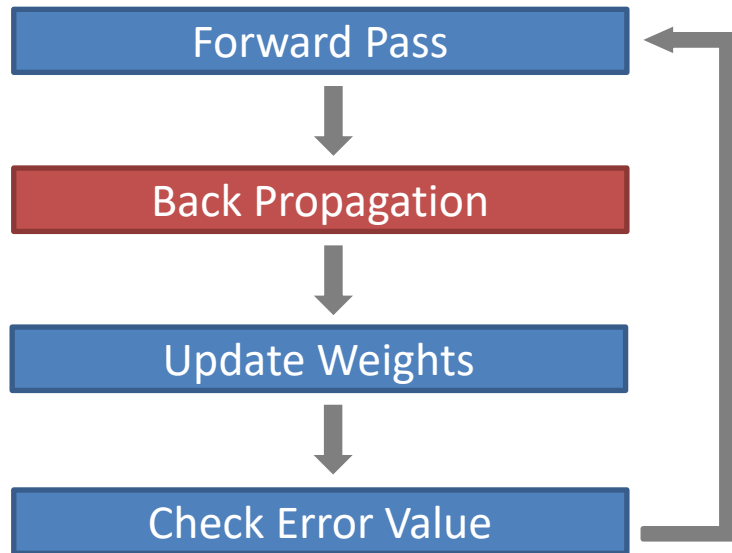
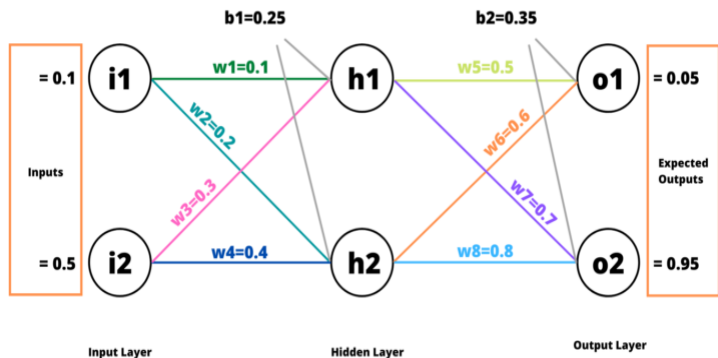
$$\frac{\partial E_1}{\partial w_1} = \frac{\partial E_1}{\partial output_{o1}} * \frac{\partial output_{o1}}{\partial sum_{o1}} * \frac{\partial sum_{o1}}{\partial output_{h1}} * \frac{\partial output_{h1}}{\partial sum_{h1}} * \frac{\partial sum_{h1}}{\partial w_1}$$

$$\frac{\partial E_2}{\partial w_1} = \frac{\partial E_2}{\partial output_{o2}} * \frac{\partial output_{o2}}{\partial sum_{o2}} * \frac{\partial sum_{o2}}{\partial output_{h1}} * \frac{\partial output_{h1}}{\partial sum_{h1}} * \frac{\partial sum_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial w_1} = 0.00159 + (-0.00049) = 0.00110.$$

$$new_w_1 = w_1 - n * \frac{\partial E_{total}}{\partial w_1} \quad new_w_1 = 0.09933$$

6. Stop when values converge.



Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

MSE with L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

MSE with L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^M W_j^2}_{\text{Regularization Term}}$$

Mean Square Log Error

$$MSLE = \frac{1}{n} \sum_{i=1}^n (\log(Y_i) - \log(\hat{Y}_i))^2$$

Huber Loss Function

$$L_{\delta}(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{for } |y - f(x)| \leq \delta, \\ \delta |y - f(x)| - \frac{1}{2}\delta^2 & \text{otherwise.} \end{cases}$$

Binary Cross Entropy – Binary Classification

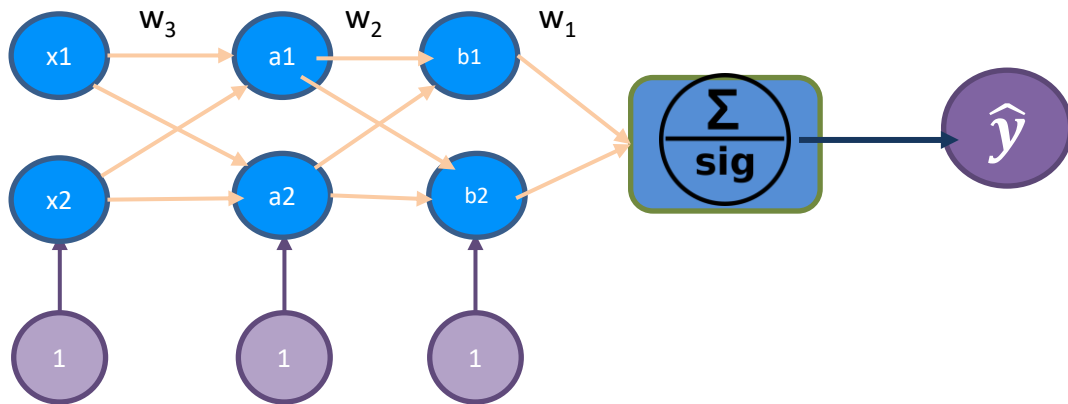
$$J = - \sum_{i=1}^N y_i \log(h_{\theta}(x_i)) + (1 - y_i) \log(1 - h_{\theta}(x_i))$$

Cross Entropy – Multi Class Classification

$$\text{cross-entropy} = - \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^k t_{i,j} \log(p_{i,j})$$

Kullback Leibler Divergence Loss (KL Loss)

$$D_{KL}(P||Q) = \begin{cases} - \sum_x P(x) \cdot \log \frac{Q(x)}{P(x)} = \sum_x P(x) \cdot \log \frac{P(x)}{Q(x)}, & \text{for discrete distributions} \\ - \int P(x) \cdot \log \frac{Q(x)}{P(x)} \cdot dx = \int P(x) \cdot \log \frac{P(x)}{Q(x)} \cdot dx, & \text{for continuous distributions} \end{cases}$$



$$\frac{\partial L}{\partial w} = \left[\frac{\partial L}{\partial b} \right] * \left[\frac{\partial b}{\partial a} \right] * \left[\frac{\partial a}{\partial x} \right]$$

Derivative of a vector w.r.t
another vector yields a
Matrix – **Jacobian**

Linear Algebra & Matrix Calculus

Batch Gradient Descent

Uses ALL training samples to calculate Error, for adjusting weights

Good for small datasets. Very time consuming with large datasets, large feature set.

Mini-Batch Gradient Descent

Uses A random sample to calculate Error, for adjusting weights

Compromise between the two and works well in both cases. Efficient use of matrix multiplication, especially using GPUs.

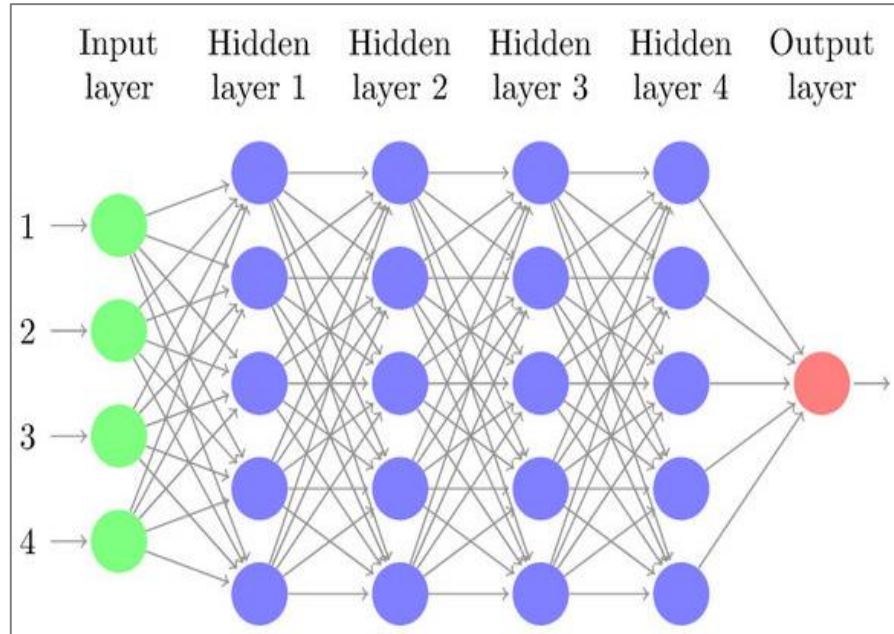
Stochastic Gradient Descent

Uses ONE training Sample to calculate Error, for adjusting weights. Also called “Online” training.

Good for very big dataset with large no of features and redundant data.

Parameters:

- Weights
- Biases



Hyper Parameters:

- Number of layers
- Number of neurons in the input, hidden and output layers
- Activation Function
- Regularization
- Learning rate
- Batch Size
- Number of epochs

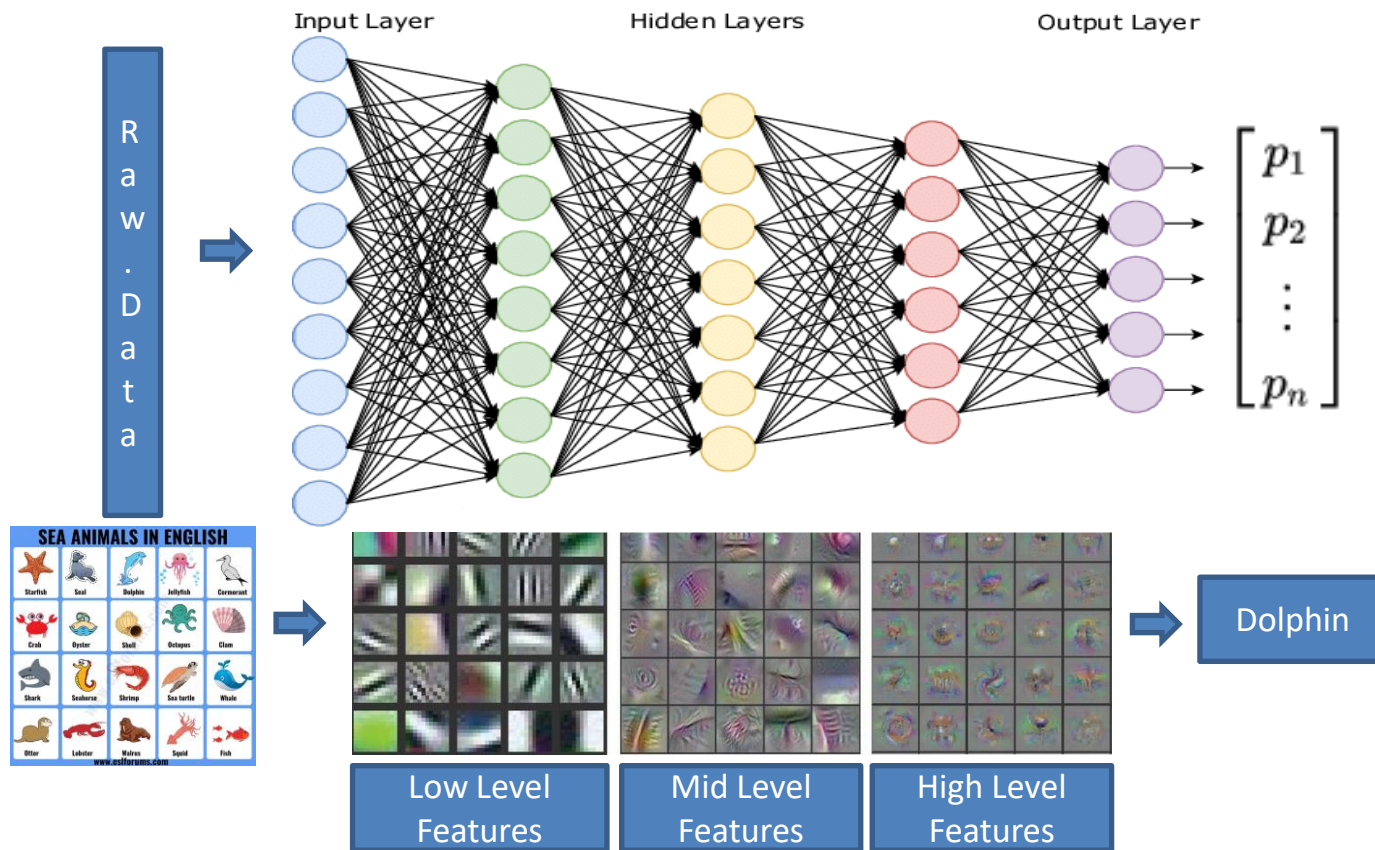
Traditional machine learning



Deep learning



CNN- Illustration of Hierarchical Feature Extraction



A mostly complete chart of

Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org

Backfed Input Cell

Input Cell

Noisy Input Cell

Hidden Cell

Probabilistic Hidden Cell

Spiking Hidden Cell

Output Cell

Match Input Output Cell

Recurrent Cell

Memory Cell

Different Memory Cell

Kernel

Convolution or Pool

Perceptron (P)



Feed Forward (FF)



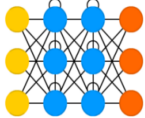
Radial Basis Network (RBF)



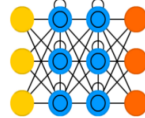
Deep Feed Forward (DFF)



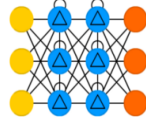
Recurrent Neural Network (RNN)



Long / Short Term Memory (LSTM)



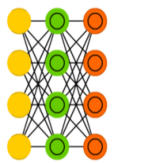
Gated Recurrent Unit (GRU)



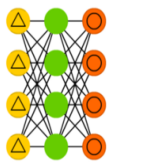
Auto Encoder (AE)



Variational AE (VAE)



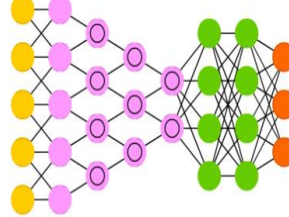
Denoising AE (DAE)



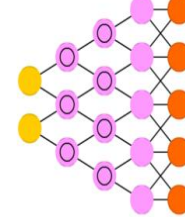
Sparse AE (SAE)



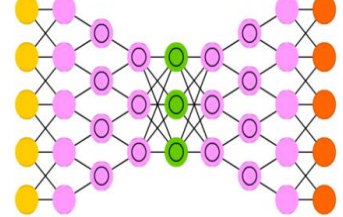
Deep Convolutional Network (DCN)



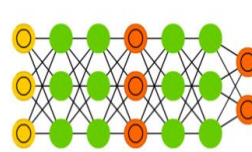
Deconvolutional Network (DN)



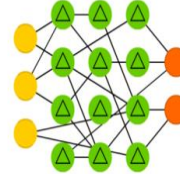
Deep Convolutional Inverse Graphics Network (DCIGN)



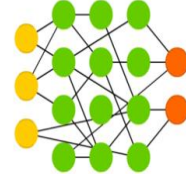
Generative Adversarial Network (GAN)



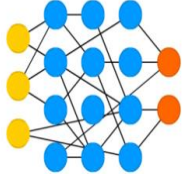
Liquid State Machine (LSM)



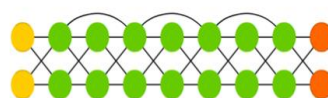
Extreme Learning Machine (ELM)



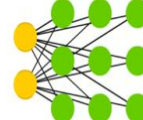
Echo State Network (ESN)



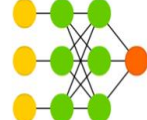
Deep Residual Network (DRN)



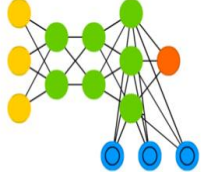
Kohonen Network (KN)



Support Vector Machine (SVM)



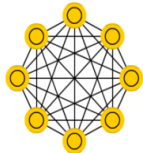
Neural Turing Machine (NTM)



Markov Chain (MC)



Hopfield Network (HN)



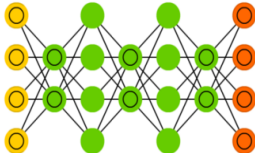
Boltzmann Machine (BM)

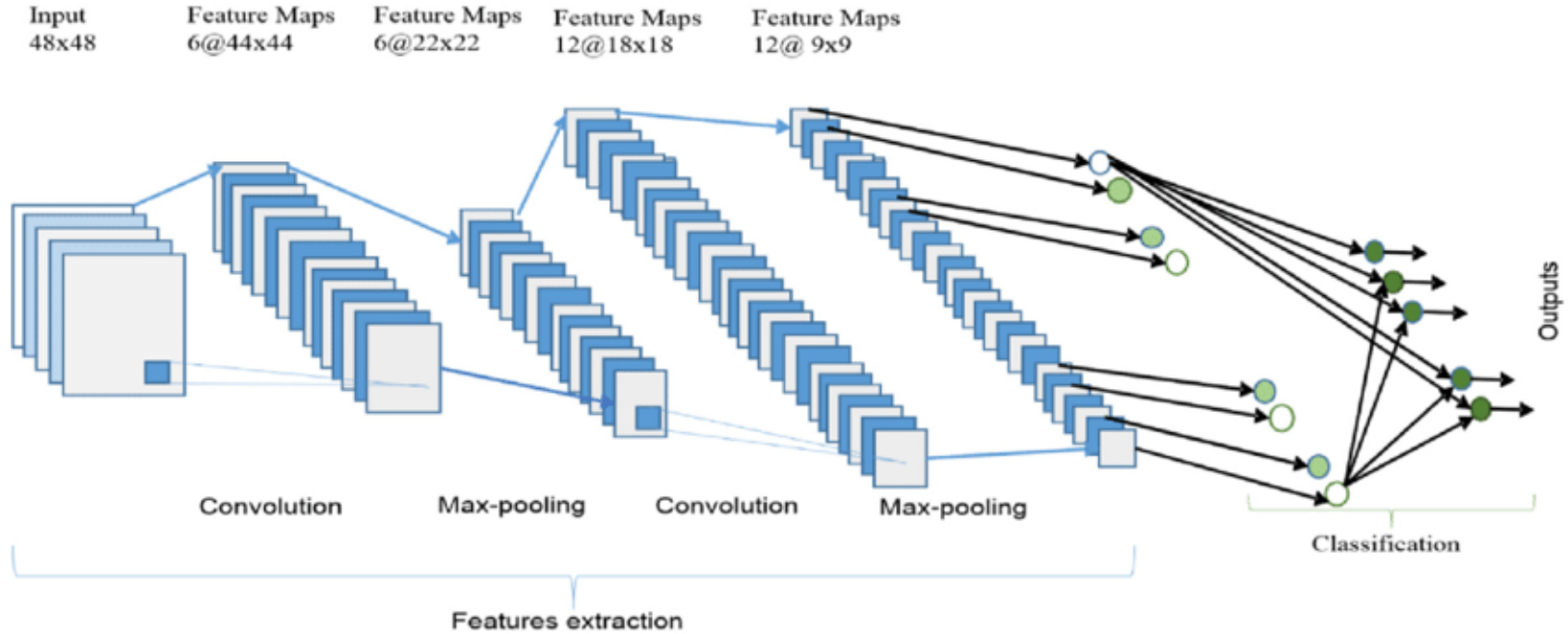


Restricted BM (RBM)



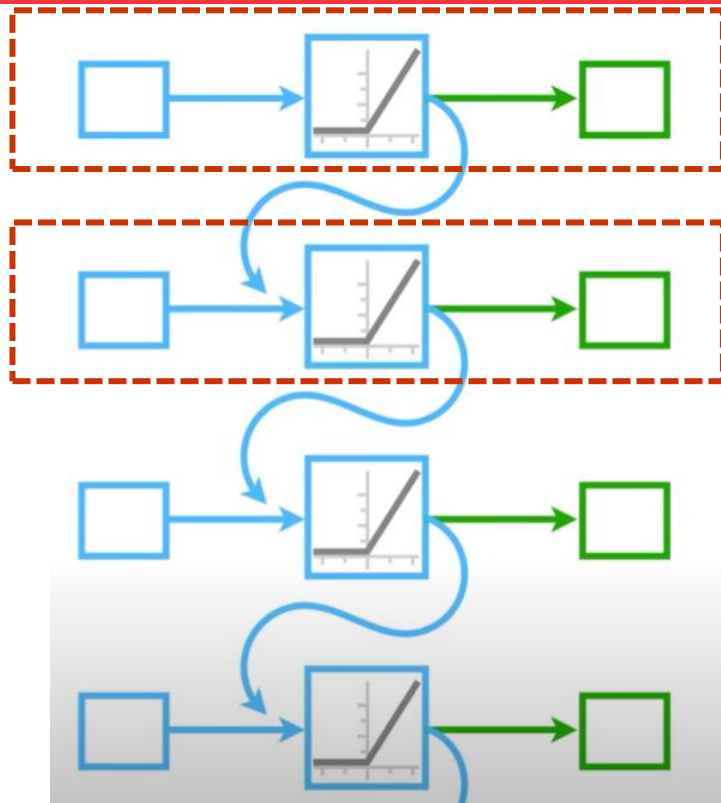
Deep Belief Network (DBN)





- Sequence of **convolution** (of filters) and **pooling** of pixel values exploit spatial locality- enforce a local connectivity pattern between neurons of adjacent layers. The learned "filters" produce the strongest response to a spatially local input pattern.
- Each filter is replicated across the entire visual field – leading to translation invariance

- RNN is a class of ANN with connections between nodes (cycle) - Output from some nodes to affect subsequent input to the same nodes.
- Exhibit temporal dynamic behavior.
- Process variable length sequences of inputs and generate variable sequence output.
- RNN can take a series of inputs with no predetermined limit on size and generate variable sequence output:
 1. Vector-Sequence Models — Image caption:
 2. Sequence-Vector Model — Sentiment analysis of a movie
 3. Sequence-to-Sequence Model - Language translation



Recurrent Neural Network – Applications

Speech recognition



“The quick brown fox jumped over the lazy dog.”

Music generation

∅



Sentiment classification

“There is nothing to like in this movie.”



DNA sequence analysis

AGCCCCTGTGAGGAACTAG



AG**CCCTGTGAGGAACT**AG

Machine translation

Voulez-vous chanter avec moi?



Do you want to sing with me?

Video activity recognition



Running

Name entity recognition

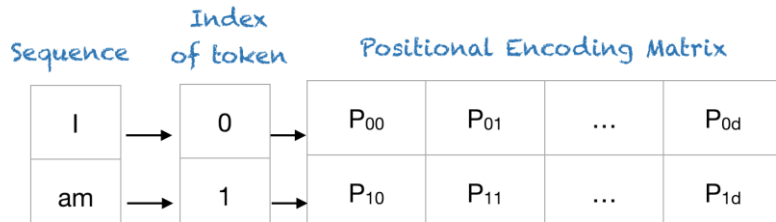
Yesterday, Harry Potter met Hermione Granger.



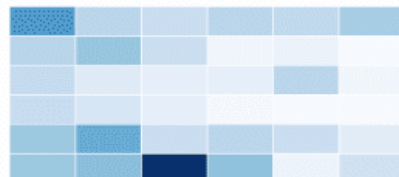
Yesterday, **Harry Potter** met **Hermione Granger** 🇧🇪

The transformer neural network is a novel architecture that aims to solve sequence-to-sequence tasks while handling long-range dependencies.

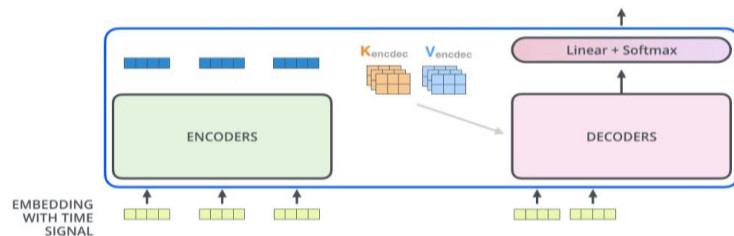
1. Positional encoding : Word embeddings are combined with their position in the sentence. This allows to input the entire sentence at once, and parallelize the process (using GPU)

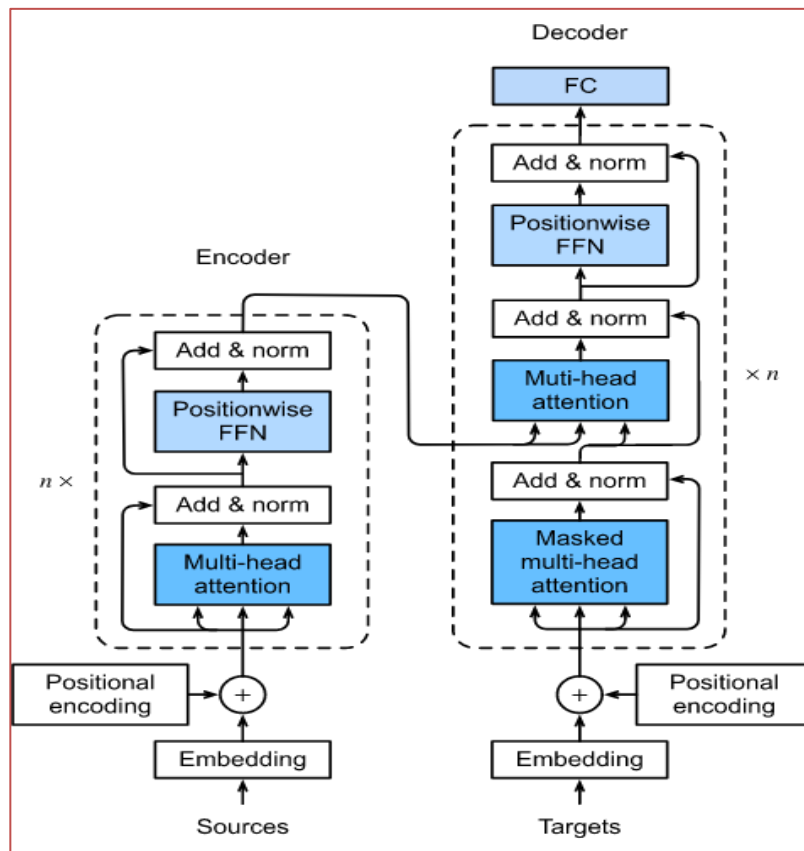


2. Self – Attention : Derive the relationship between each word with all other words in the input (using Query, Key and Value mechanism)



3. Decoding: The decoder generates the output sentence by defining the conditional probability distribution of a target sequence given a contextualized encoding sequence.





- BERT :
 - Bidirectional Encoder Representations from Transformers is a transformer-based machine learning technique for natural language processing.
 - BERT was created and published in 2018 by Google.
 - BERT is a deeply bidirectional, unsupervised language representation, pre-trained using only a plain text corpus.
- GPT-3 :
 - **Generative Pre-trained Transformer 3** is a language model that uses deep learning to produce human-like text.
 - GPT-3, which was introduced in May 2020, by OpenAI.
 - The architecture is a standard transformer network (with a few tweaks) with 175 billion parameters.

As aliens entered our planet, ...

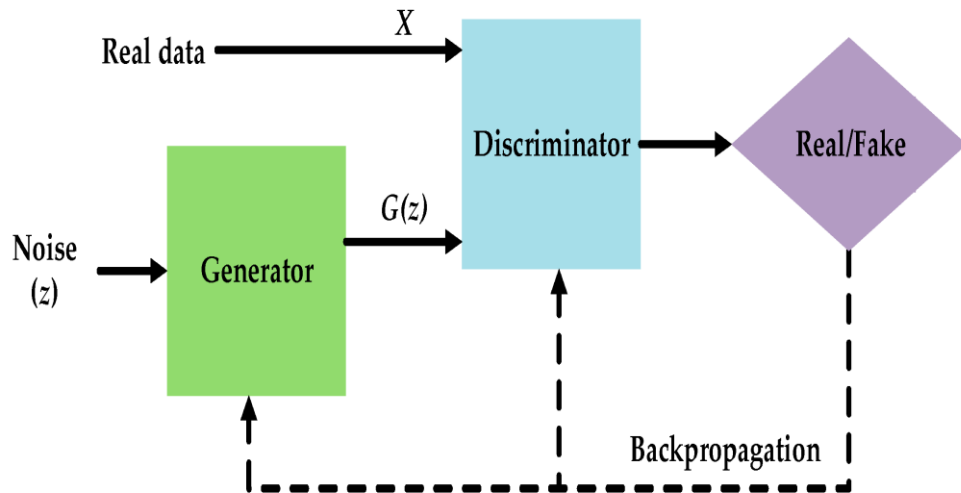
and began to colonize the Earth, certain group of extra-terrestrial

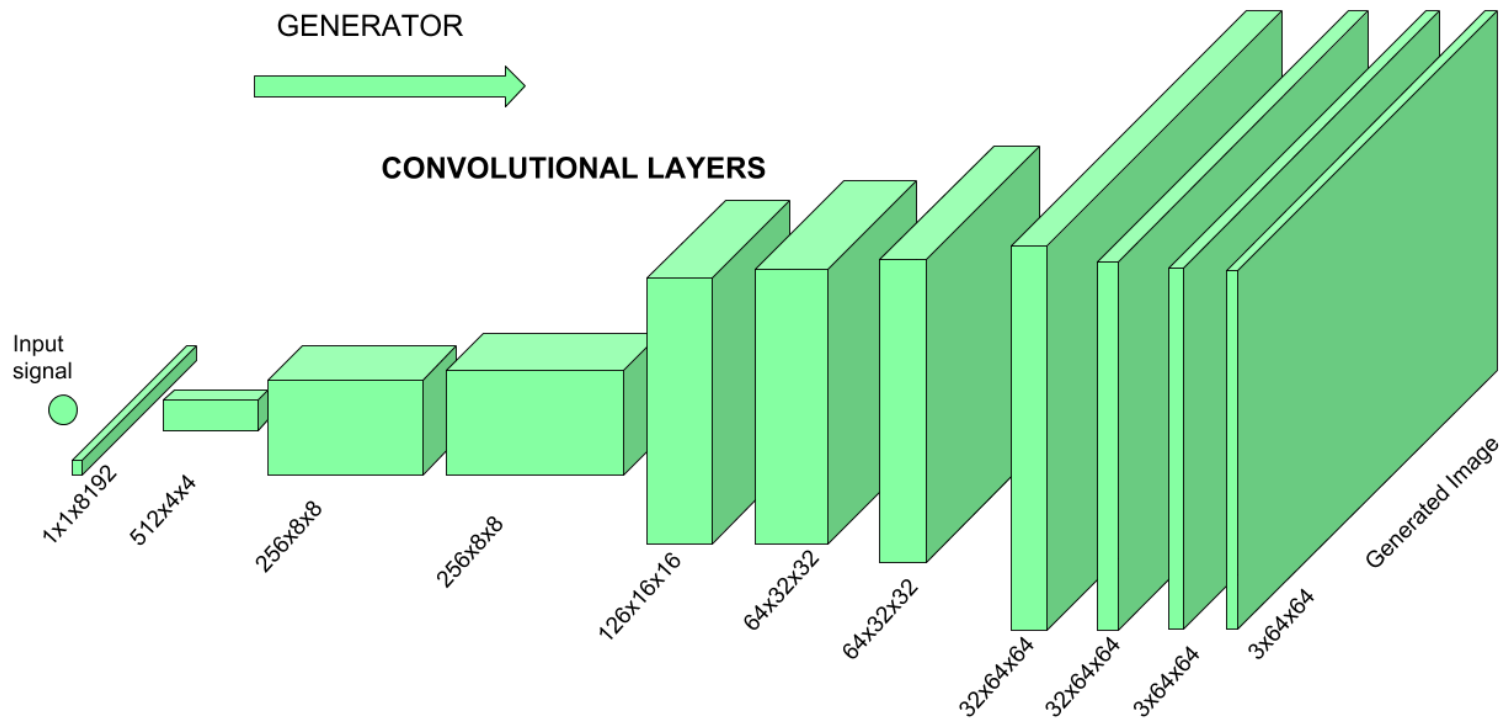
RNN

GRU / LSTM

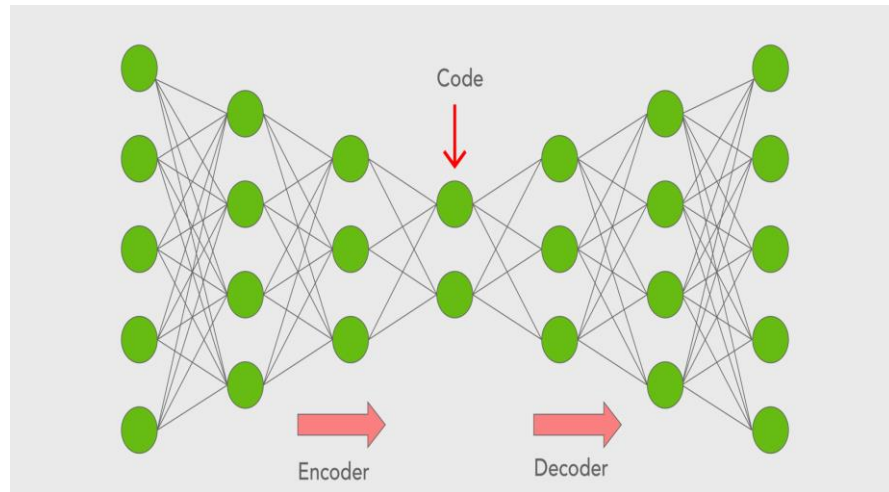
Transformer

- GAN is a class of ML frameworks where two neural networks contest with each other in the form of a zero-sum game.
- Given a training set, this technique learns to generate new data with the same statistics as the training set.
- For example, a GAN trained on photographs can generate new photographs that look at least superficially authentic to human observers, having many realistic characteristics.



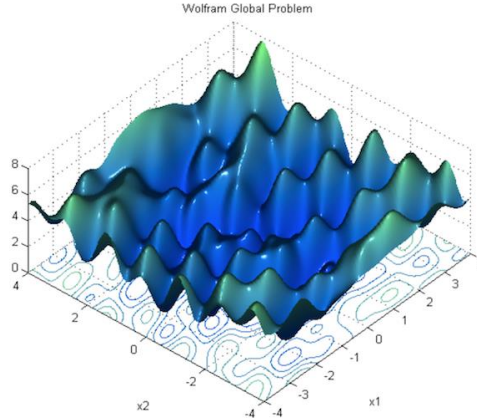


- Autoencoders operate by taking in data, compressing and encoding the data, and then reconstructing the data from the encoding representation.
- The model is trained until the loss is minimized and the data is reproduced as closely as possible.
- Through this process, an autoencoder can learn the important features of the data
- Used for dimensionality reduction, data denoising, feature extraction, image generation, sequence to sequence prediction, and recommendation systems.





Large Dataset



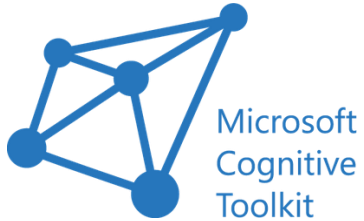
Global Minima



Resource & Time

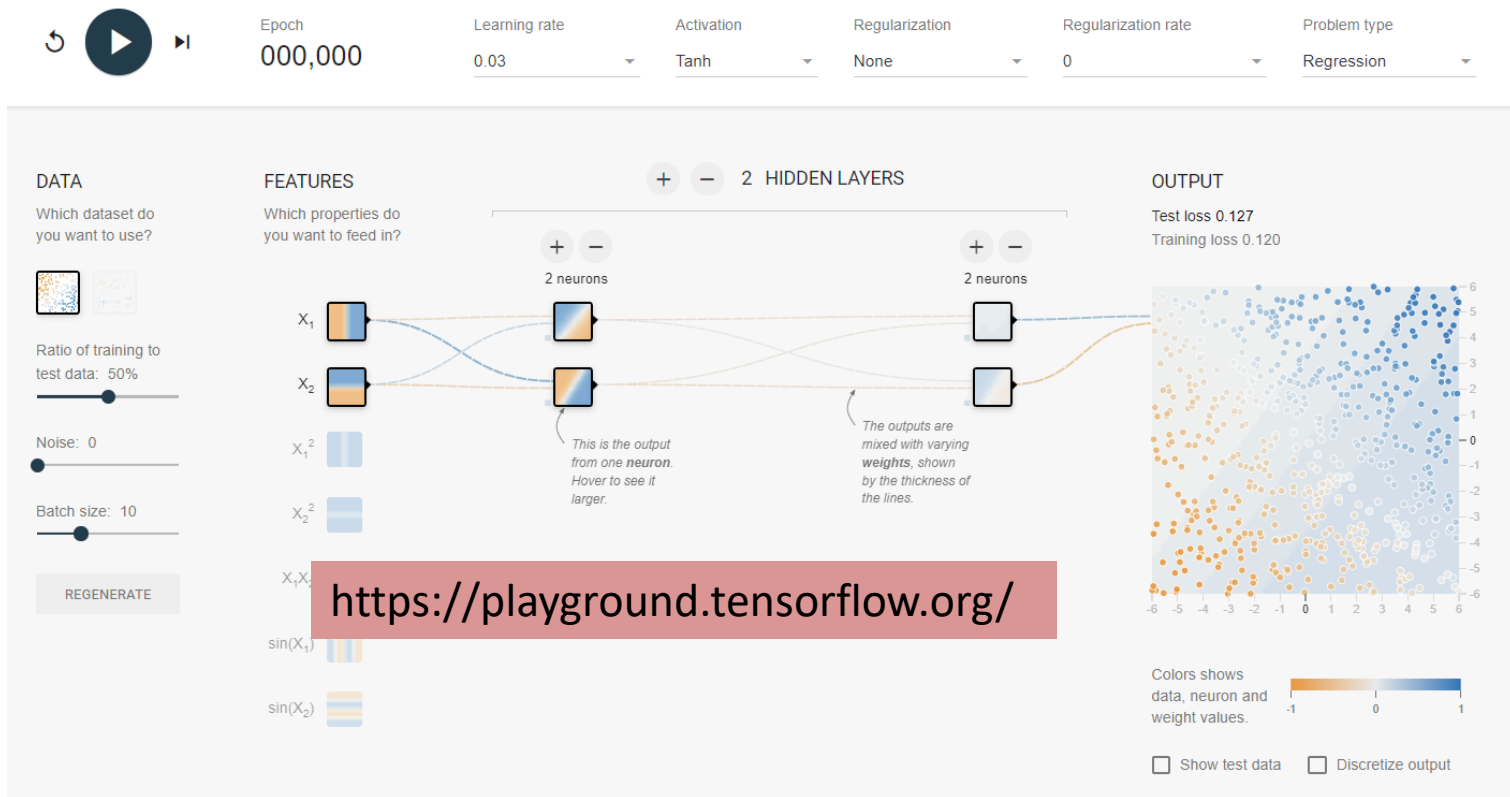


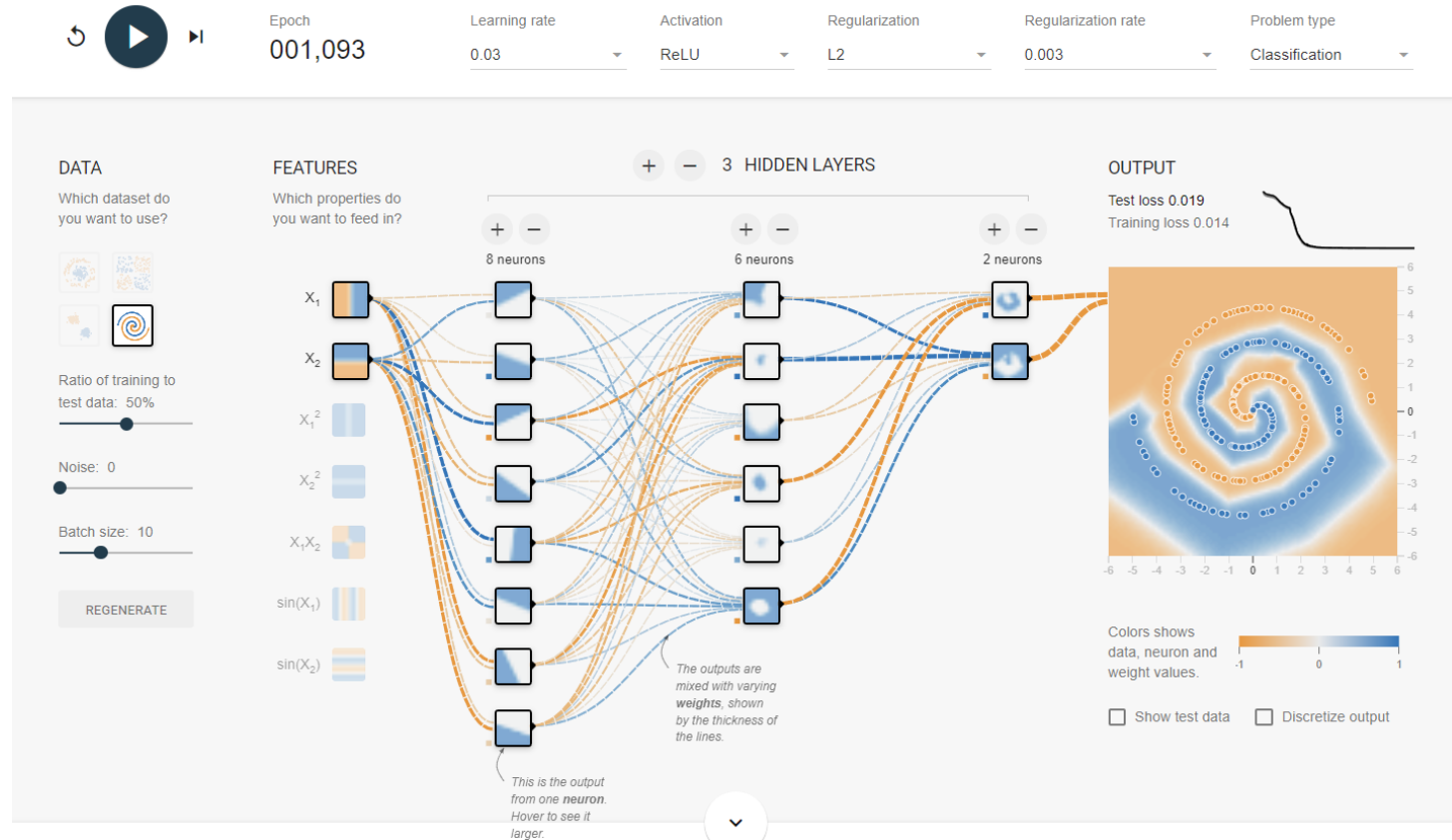
Caffe



DL4J

theano









Thank You!